

UNIVERZITA PARDUBICE

FAKULTA ELEKTROTECHNIKY A
INFORMATIKY

BAKALÁŘSKÁ PRÁCE

2025

Egor Razboinikov

Univerzita Pardubice
Fakulta Elektrotechniky a Informatiky

Digitalizace evidence pracovní doby
Bakalářská práce

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení:	Egor Razboinikov
Osobní číslo:	I21219
Studijní program:	B0688A140009 Informační technologie
Téma práce:	Digitalizace evidence pracovní doby
Zadávací katedra:	Katedra informačních technologií

Zásady pro vypracování

Dokument evidence pracovní doby vyučujícího vyžaduje provázanost rozvrhu s úvazkem vyučujícího při respektování času přestávek. Cílem bakalářské práce je navrhnout, vytvořit, odzkoušet a předat funkční multiplatformní aplikaci jejíž vstupy jsou: rozvrh vyučujícího, čas na oběd, plán dovolených, státní svátky, školní volna a další. Požadovaným výstupem je PDF dokument (tabulka) se všemi náležitostmi potřebnými pro odevzdání. Grafické rozhraní poskytne pohodlné zadávání, umožní dodatečné ruční modifikace a zobrazování náhledu před vygenerováním PDF dokumentu.

Rozsah pracovní zprávy: **min. 30 stran**
Rozsah grafických prací:
Forma zpracování bakalářské práce: **tištěná**

Seznam doporučené literatury:

PŠENČÍKOVÁ, J. Algoritmizace. Kralice na Hané, Computer Media, 2009.
MAREŠ, Martin a VALLA, Tomáš. Průvodce labyrintem algoritmů 2. vydání. Praha: CZ.NIC, 2022. ISBN 978-80-88168-63-8
PECINOVSKÝ, Rudolf. OOP: naučte se myslet a programovat objektově. Brno: Computer Press, 2010. ISBN 978-80-251-2126-9.

Vedoucí bakalářské práce: **Ing. Radek Matoušek**
Katedra informačních technologií

Datum zadání bakalářské práce: **15. prosince 2024**
Termín odevzdání bakalářské práce: **16. května 2025**

L.S.

prof. Ing. Petr Doležel, Ph.D. v.r.
děkan

Ing. Jan Panuš, Ph.D. v.r.
vedoucí katedry

V Pardubicích dne 28. února 2025

Prohlašuji:

Práci s názvem Digitalizace evidence pracovní doby jsem vypracoval samostatně. Veškeré literární prameny a informace, které jsem v práci využil, jsou uvedeny v seznamu použité literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 7/2019 Pravidla pro odevzdávání, zveřejňování a formální úpravu závěrečných prací, ve znění pozdějších dodatků, bude práce zveřejněna prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 9. 5. 2025

Egor Razboinikov v.r.

PODĚKOVÁNÍ

Tady chtěl bych poděkovat svému vedoucímu Ing. Radek Matoušek za hezké vedení bakalářské práce a rychle odpovědi. Také rád bych poděkoval svým rodičům a kamarádům za jejich podporu po celou dobu studia.

ANOTACE

Cílem této bakalářské práce je návrh, implementace a ověření funkčnosti multiplatformní aplikace pro digitalizaci evidence pracovní doby vyučujících. Aplikace umožňuje automatizované zpracování dat, jako jsou rozvrh hodin vyučujícího, stanovené přestávky na oběd, plánované dovolené, státní svátky a školní volna. Výsledným výstupem je detailní PDF dokument, který splňuje formální požadavky fakulty a lze jej snadno upravovat prostřednictvím intuitivního grafického rozhraní. V práci je popsána použitá metodika, výběr vhodných technologií a shrnuty výsledky praktického ověření aplikace.

KLÍČOVÁ SLOVA

C#, .NET, AVALONIA UI, MVVM, AXAML, desktopová aplikace, rozvrh hodin, evidence pracovní doby, generování PDF, automatizace.

TITLE

Digitalization of Working Time Records

ANNOTATION

The aim of this bachelor thesis is to design, implement, and verify a functional cross-platform application for digitizing teachers' working time records. The application automates data processing, including teachers' timetables, designated lunch breaks, vacation schedules, public holidays, and school breaks. The output is a detailed PDF document that meets the faculty's formal submission requirements and can be easily modified via an intuitive graphical interface. The thesis describes the applied methodology, technology selection, and summarizes results of the practical evaluation of the application.

KEYWORDS

C#, .NET, WPF, MVVM, AVALONIA, desktop application, timetable, working time tracking, PDF generation, automation.

OBSAH

SEZNAM ILUSTRACÍ	11
SEZNAM ZKRATEK A ZNAČEK	12
TERMINOLOGIE	14
ÚVOD	15
1 TEORETICKÁ ČÁST	16
1.1 Pravidla evidence pracovní doby	16
1.2 Pravidla a teoretické zásady algoritmu vyrovnávání	16
1.3 Architektura a vzory	18
1.3.1 Co je architektura softwaru	18
1.3.2 MVVM – Model–View–ViewModel	18
1.3.3 MVC – Model–View–Controller	19
1.3.4 Monolit, modulární monolit a mikroslužby	20
1.4 Programovací jazyky	20
1.4.1 Jazyk C#	21
1.4.2 Jazyk SQL	22
1.5 Framework Avalonia UI	23
1.6 Vývojové prostředí	24
1.6.1 Microsoft Visual Studio	24
1.6.2 SQLiteStudio	24
1.7 Knihovny	25
2 PRAKTICKÁ ČÁST	26
2.1 Programovací jazyky	26
2.1.1 Jazyk C#	26
2.1.2 Jazyk XAML	27
2.2 Struktura aplikace	28
2.4 Souborová struktura aplikace	29

2.4.1 Složka Behaviors	30
2.4.2 Složka Controls.....	31
2.4.3 Složka Converters	31
2.4.4 Složka Data	31
2.4.5 Složka Helpers	32
2.4.6 Složka Messages	32
2.4.7 Složka Models.....	33
2.4.8 Složka Services	33
2.4.9 Složka ViewModels	33
2.4.10 Složka Views	33
2.4.11 Složka Kořenové soubory projektu.....	33
2.5 Struktura databáze.....	33
2.5.1 Přehled tabulek	34
2.5.2 Bez cizích klíčů: záměr a důvody	35
2.5.3 Shrnutí rozhodnutí	35
2.6 Uživatelské rozhraní aplikace	36
2.6.1 Hlavní stránka aplikace.....	36
2.6.2 Dialogové okno pro vytvoření události	38
2.6.3 Dialogové okno pro změnu události	39
2.6.4 Vizuální prezentace pro den	40
2.6.5 Vizuální prezentace pro týden	41
2.6.6 Vizuální prezentace pro měsíc	42
2.6.7 Globální nastavení	43
2.6.8 Ukázka PDF	45
2.7 Popsání algoritmů	45
2.7.1 Popis algoritmu BalanceDays	45
2.7.2 Popis algoritmu ze služby pro výpočet pracovní doby	49

ZÁVĚR	57
POUŽITÁ LITERATURA	58

SEZNAM ILUSTRACÍ

Nebyla nalezena položka obsahu.

Obrázek 1: Architektura MVVM.....	19
Obrázek 2 Architektura MVC.....	20
Obrázek 3: Souborová struktura aplikace	30
Obrázek 4 Ukázka kódu třídy AppDbContex.....	32
Obrázek 5 Levý panel aplikace.....	37
Obrázek 6 Pravý panel aplikace.....	38
Obrázek 7 Dialogové okno pro vytvoření.....	39
Obrázek 8 Dialogové okno pro změnu	40
Obrázek 9 Vizuální prezentace pro den	41
Obrázek 10 Vizuální prezentace pro týden.....	42
Obrázek 11 Vizuální prezentace pro měsíc	43
Obrázek 12 Globální nastavení.....	44
Obrázek 13 Ukázka PDF	45

SEZNAM ZKRATEK A ZNAČEK

EPD – evidence pracovní doby

FPD – fond pracovní doby

D – dovolená

PC – pracovní cesta

N – nemoc

OČR – ošetřování člena rodiny

L – lékař

S – svátek

CSV – Comma-Separated Values

PDF – Portable Document Format

GUI – Graphical User Interface

STAG – Studijní agenda

MVC – Model – View – Controller

OS – operační systém

DB – Databáze

DI – Dependency Injection

IoC – Inversion of Control

EF Core – Entity Framework Core

FK – Foreign Key

UI – User Interface

JIT – Just-In-Time

MVVM – ModelView – View – Model

ORM – Object–Relational Mapping

PDF – Portable Document Format

UI – User Interface

CLR – Common Language Runtime

XAML – eXtensible Application Markup Language

AXAML – Avalonia eXtensible Application Markup Language

RI – Run-Time Information

CI/CD – Continuous Integration / Continuous Deployment

TERMINOLOGIE

Avalonia UI: Multiplatformní framework pro vývoj uživatelského rozhraní aplikací, který umožňuje tvorbu vizuálně konzistentních aplikací běžících na různých operačních systémech, jako jsou Windows, macOS a Linux, za pomoci XAML a programovacího jazyka C#.

Framework: Sada nástrojů, knihoven a standardizovaných postupů, která usnadňuje proces vývoje softwaru poskytováním struktury a základních funkcionalit, umožňujících rychlou implementaci úloh bez nutnosti opakování vytváření stejného kódu.

Relační databáze: Typ databáze ukládající informace v tabulkách s jasně definovanými vztahy (např. vazby jedna k jedné, jedna k mnoha), umožňující efektivní ukládání, vyhledávání a správu dat.

Programovací paradigma (MVVM): Model-View-ViewModel představuje návrhový vzor programování, který jasně odděluje uživatelské rozhraní (View), datovou logiku (Model) a komunikační logiku (ViewModel), což vede k lepší udržitelnosti a škálovatelnosti aplikace.

Uživatelské rozhraní (UI): Vizuální a interaktivní část aplikace, navržená pomocí frameworku Avalonia UI, která umožňuje uživatelům jednoduše komunikovat s programem prostřednictvím grafických prvků, jako jsou tlačítka, formuláře, menu a další ovládací prvky.

Integrované vývojové prostředí (IDE): Softwarový nástroj kombinující editor kódu, kompilátor, debugger a nástroje pro správu verzí, používaný při vývoji aplikací.

Systémy pro správu verzí (VCS): Nástroje umožňující sledování změn kódu, uchování historie verzí, spolupráci mezi vývojáři a obnovu předchozích verzí kódu v případě potřeby.

ÚVOD

Evidence pracovní doby je formální dokument, který vyučující odevzdává pravidelně za jednotlivé měsíce. Správné rozložení hodin podle rozvrhu, přestávek a dalších pravidel není vždy jednoduché. Proto je vhodné celý proces digitalizovat a co nejvíce automatizovat.

Cílem této bakalářské práce je navrhnout, vytvořit, otestovat a předat funkční multiplatformní aplikaci, která vyučujícímu pomůže připravit EPD. Vstupem je pouze rozvrh vyučujícího exportovaný ze systému STAG ve formátu CSV. Aplikace z těchto dat sestaví přehled pracovní doby a vygeneruje výstupní dokument ve formátu PDF připravený k odevzdání. Součástí řešení je jednoduché grafické rozhraní, které umožní pohodlné zadávání, náhled a ruční doplnění či úpravu informací, které v rozvrhu chybí, například dovolená, služební cesta nebo nemoc. Program je navržen tak, aby byl použitelný na systémech Windows i Linux bez virtualizace.

Navržený algoritmus automaticky převádí data z rozvrhu vyučujícího (CSV) do podoby, která odpovídá požadavkům na evidence pracovní doby (EPD – evidence pracovní doby). V povolených mezích umí upravit časy příchodu, odchodu a polední přestávky, hlídá kolize s výukou a vyrovnává nerovnoměrnosti mezi dny a týdny. Po zpracování jsou data v konzistentním a formálně správném stavu a lze z nich přímo vygenerovat tabulku připravenou k exportu do formátu PDF. Podrobná pravidla i přesná struktura výstupní tabulky jsou popsány v samostatné kapitole.

Motivací práce je zjednodušit administrativu vyučujícího a ušetřit čas. Automatické zpracování z rozvrhu a přehledné grafické rozhraní mají vyučujícímu umožnit rychlou kontrolu a drobné doladění detailů. Přínosem je také jednotný a formálně správný výstup v PDF.

Teoretická část popisuje použitý model architektury, klíčové algoritmy pro rozkládání pracovní doby a pravidla, která aplikace hlídá. Praktická část se zaměřuje na analýzu požadavků, návrh uživatelského rozhraní, strukturu dat, import rozvrhu ze systému STAG, generování PDF a testování na reálných scénářích. V závěru jsou zhodnoceny dosažené cíle a návrhy dalšího rozvoje.

1 TEORETICKÁ ČÁST

1.1 Pravidla evidence pracovní doby

Tato podkapitola sjednocuje pravidla. Výkaz má odpovídat internímu vzoru EPD. Používáme jednoduché formulace a žádné vzorce.

Každý pracovní týden musí v součtu odpovídat standardní týdenní pracovní době. U plného úvazku jde o 40 hodin. U zkrácených úvazků platí hodnota z pracovní smlouvy.

Pokud je v týdnu státní svátek nebo jiný den pracovního klidu, snižuje se požadovaný týdenní součet o 8 hodin za každý takový den. Jeden svátek tedy znamená 32 hodin za týden.

Když má týden méně než pět pracovních dnů, například na začátku nebo na konci měsíce, počet hodin odpovídá počtu pracovních dnů. Každý takový den se počítá jako 8 hodin.

Hodiny nad běžný den zapisujeme do sloupce „Odpracováno“. Sloupec „Přesčas“ nevyplňujeme. Přesčas je až práce nad týdenní limit. Napracování volna není přesčas. Součet ve sloupci „Odpracováno“ za týden musí vždy odpovídat požadované týdenní hodnotě.

Začátek práce, přestávka a konec práce musí odpovídat rozvrhu ve STAGu. Pokud na sebe výuka navazuje, přestávka je na jejich rozhraní. V docházce obvykle uvádíme 8 hodin práce a 30 minut přestávky.

Ve všední den vykazujeme služební cestu jako běžnou osmihodinovou přítomnost. Cestovní náhrady do EPD nepatří a řeší je cestovní příkaz. Práce na služební cestě o víkendu se do EPD nezapisuje.

Do sloupce „Neodpracováno“ zapisujeme součet hodin z dovolené, z nemoci, z ošetřování člena rodiny nebo z návštěvy lékaře. Státní svátky jsou ve výkazu označeny předem.

Součet hodin za měsíc musí odpovídat měsíčnímu fondu pracovní doby (FPD). Kontrolu provedeme tak, že sečteme odpracované hodiny, případné položky v „Přesčas“ a neodpracované hodiny. Výsledek se musí rovnat FPD. Ve vzorové tabulce to hlídají souhrnné buňky v řádku „Celkem“.

1.2 Pravidla a teoretické zásady algoritmu vyrovnávání

1.2.1 Teorie algoritmu

Tato část popisuje, co má algoritmus dělat a proč. Cílem je převést rozvrh vyučujícího do podoby vhodné pro EPD. Výstup musí splnit týdenní i měsíční požadavky. Nesmí obsahovat

vnitřní rozpory. Musí respektovat povolené překryvy. Zároveň se má co nejméně odchylovat od skutečného rozvrhu.

Algoritmus je stabilní a předvídatelný. Rozvrh ze souboru je pevný a algoritmus jej nikdy nemění ani neposouvá. Stejně vstupy vždy dávají stejný výstup. Malá změna v nepřítomnostech nebo v nastavení vyvolá jen malou a místní změnu ve výkazu. Díky tomu je výsledek snadno kontrolovatelný.

Všechny zásahy musí být průhledné a dohledatelné: je jasné, proč vznikly a jak ovlivnily výpočet

Algoritmus vychází z rozvrhu exportovaného ze Studijní agendy ve formátu CSV. Nepřítomnosti a pracovní cesty zadává uživatel. Algoritmus je sám nevytváří.

1.2.2 Pravidla algoritmu

Algoritmus používá dané pořadí typů událostí (stavů dne). Nejvýše jsou svátky a jiné dny pracovního klidu. Poté následují nepřítomnosti: dovolená, nemoc, ošetřování člena rodiny a návštěva lékaře. Dále je pracovní cesta. Nakonec je běžná práce podle rozvrhu. Typ s vyšší prioritou přepisuje typ s nižší prioritou. Pokud dojde ke kolizi (časový překryv dvou událostí ve stejném čase), uplatní se událost s vyšší prioritou podle svého typu. Nepřítomnost tak může překrýt výuku.

Platí tvrdá pravidla. Když se ve stejném čase potká událost práce a událost s vyšší prioritou, zapíše se jen tato vyšší priorita a překrývající se práce se do „Odpracováno“ nepočítá. Když jsou v jednom dni události v různých časech, výkaz je rozdělí do dvou bloků: odpracovaný čas do „Odpracováno“ a neodpracovaný čas do „Neodpracováno“ s jasnou značkou důvodu. Bloky se nesmí překrývat a musí tvořit souvislé úseky. Po zaokrouhlení na společnou časovou mřížku musí zůstat konzistentní. Současně platí pravidla z části 1.1: týdenní součty, zacházení s neúplnými týdny a týdny se svátky a měsíční fond pracovní doby.

Měkká pravidla říkají, že změny mají být malé a rovnoměrné. Přednost má úprava délky a umístění polední přestávky. Posuny příchodu a odchodu přicházejí až potom. Pokud existuje více stejně dobrých řešení, vybírá se to s menší odchylkou od rozvrhu.

Povolené zásahy jsou omezené. Algoritmus smí jemně posouvat příchod a odchod v mezích, které nastaví uživatel, a smí měnit délku a umístění přestávky. Události typu nepřítomnost a služební cesta jsou nepřesouvateľné (algoritmus je neposouvá, nezkracuje ani nerozděluje).

Rozhodování má dané pořadí. Nejdříve se uplatní tvrdá pravidla. Poté se zachovají časy výuky, pokud je nepřepisuje událost s vyšší prioritou. Následně se upraví přestávky. Až nakonec se jemně posunou okraje dne. Pokud je to nutné pro měsíční fond, lze vyrovnat hodiny mezi neúplnými týdny na začátku a na konci měsíce. Plné týdny se mění až jako poslední.

Korektnost vůči měsíčnímu fondu je povinná. Součet hodin za měsíc se musí rovnat FPD. Plné týdny jsou uzavřené: úpravy se dělají jen uvnitř daného týdne (přestávky, okraje dne). Hodiny se mezi týdny nepřesouvají. Neúplné týdny na okrajích měsíce (na začátku a na konci) je možné posuzovat společně jako jeden logický celek. Mezi těmito dvěma neúplnými týdny lze vyrovnat rozdíly, aby vyšel měsíční fond. Ostatní týdny se do vyrovnání nezapojují. Všechny změny musí zohledňovat typy událostí a přísná pravidla a nesmí způsobovat konflikty. (časový překryv dvou událostí ve stejném čase). Rozvrh se nemění.

Platí zásada bezpečného selhání. Pokud nelze splnit všechna pravidla současně, algoritmus se zastaví v nejbližším konzistentním stavu. Nikdy nevytvoří hodiny, které by porušily tvrdá pravidla.

1.3 Architektura a vzory

Aplikace je desktopová a multiplatformní. Běží lokálně, má grafické rozhraní, umí importovat CSV a exportovat PDF. Nepotřebuje server ani skládání více síťových služeb. Proto volím modulární monolit: jeden spustitelný celek, uvnitř jasně oddělené moduly — import CSV, algoritmus vyrovnávání, generátor PDF a GUI. Tato volba snižuje provozní režii, zjednodušuje testování a urychluje vývoj [17][19].

1.3.1 Co je architektura softwaru

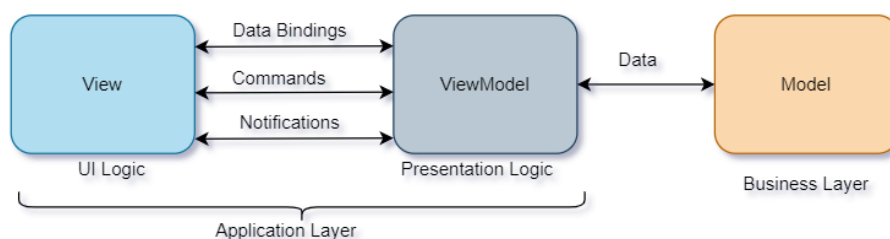
V této práci chápu architekturu softwaru jako soubor rozhodnutí o struktuře a chování systému. Architektura určuje, z jakých komponent se systém skládá, jaká mají rozhraní a jak spolu komunikují (volání rozhraní, datové toky). Dále vymezuje rozdělení odpovědností mezi komponentami a způsob, jak se zajistí požadované vlastnosti systému, jako je modifikovatelnost, spolehlivost a bezpečnost. Toto pojetí odpovídá definicím Software Engineering Institute i běžné praxi v softwarovém inženýrství.[13]

1.3.2 MVVM – Model–View–ViewModel

MVVM je architektonický vzor pro aplikace s deklarativním UI (XAML/AXAML). Odděluje zobrazení (View) od prezentační logiky (ViewModel) a od doménových dat a služeb (Model). Hlavními mechanismy jsou datové vazby (data binding) a příkazy (commands): změna stavu

ve ViewModelu se automaticky projeví ve View a akce uživatele ve View spouští příkazy ve ViewModelu. Přehled vztahů ukazuje *Obrázek 1*. Díky tomuto rozdělení je návrh testovatelný a má nízkou míru provázanosti [3][4].

V této práci vzor MVVM uplatňuji takto: obrazovky píšu v Avalonia UI (AXAML) a ponechávám je čistě prezentační. Veškerá logika je ve třídách ViewModel, které lze testovat bez běžícího UI. Pro reaktivní stav a příkazy používám ReactiveUI, které cílí na MVVM a podporuje deklarativní práci se stavem [4][12]. View komunikuje s ViewModelem pouze přes datové vazby a příkazy; k Modelu nepřistupuje přímo. ViewModel pracuje s Modelem přes rozhraní. Tím zůstává jasně oddělené Model–View–ViewModel a návrh je dobře testovatelný [3][4].

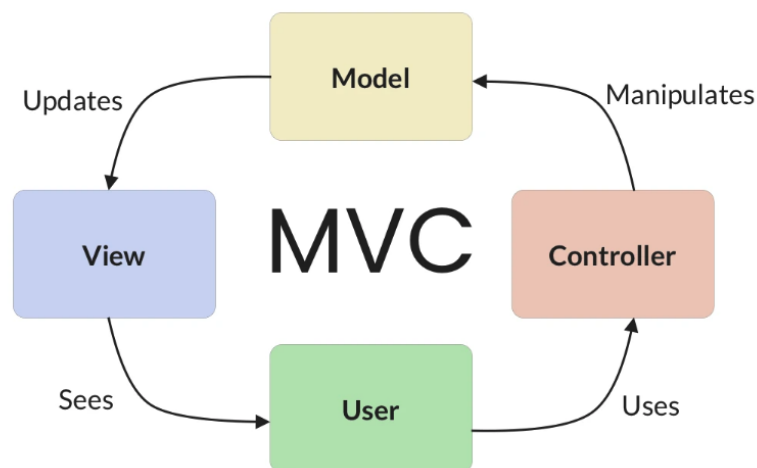


Obrázek 1: Architektura MVVM

1.3.3 MVC – Model–View–Controller

Pro srovnání stručně uvádím i architekturu MVC a důvod, proč zde volím MVVM. MVC dělí aplikaci na Model, View a Controller. Controller přijímá vstup uživatele, vybere akci a určí, který View se vykreslí. Ve webovém prostředí (např. ASP.NET MVC) jde o ověřený přístup. Ve vrstvě prezentace poskytuje jasné oddělení odpovědností [11].

U desktopových XAML aplikací (Avalonia, WPF, MAUI) je vhodnější MVVM. XAML frameworky totiž nativně podporují datové vazby a příkazy. Tyto mechanismy v MVC obvykle zajišťuje Controller ručně. MVVM je proto přirozenější, vede k menší provázanosti a k lepší testovatelnosti [3][11].



Obrázek 2 Architektura MVC

1.3.4 Monolit, modulární monolit a mikroslužby

Monolitická aplikace je jeden nasaditelný celek (jeden proces/binárka). Uvnitř může mít vrstvy a knihovny, navenek se ale nasazuje jako jedna jednotka. Pro menší až středně velké systémy jde o běžnou a srozumitelnou volbu.[17]

Modulární monolit je také jeden nasaditelný celek, ale uvnitř má přísně oddělené moduly s jasnými rozhraními (např. „Import CSV“, „Algoritmus“, „PDF“, „GUI“). Moduly spolu komunikují pouze přes tato rozhraní; přímé obcházení se nepovoluje. Díky tomu je kód lépe udržitelný a později lze jednotlivé moduly případně vyčlenit do samostatných služeb, pokud to projekt vyžaduje

Mikroslužby rozkládají systém na více samostatně nasaditelných služeb. Každá má vlastní životní cyklus a může se škálovat nezávisle. Tento přístup přináší vyšší složitost: síťovou komunikaci, řešení konzistence, observabilitu a náročnější nasazování a provoz.[18][20]

U nového projektu je často rozumné začít dobře strukturovaným modulárním monolitem. Jakmile se objeví prokazatelné důvody (např. potřeba nezávislého škálování nebo silné hranice týmů), lze vybrané moduly převést na mikroslužby. Tak se vyhneme předčasné distribuované složitosti.[19]

1.4 Programovací jazyky

Tato kapitola zdůvodňuje volbu jazyka C# na platformě .NET. Ukáže, jak tato volba podporuje multiplatformní desktopovou aplikaci s grafickým rozhraním, a stručně představí rozumné alternativy. Cílem je doložit, že C#/.NET splňuje požadavky této práce: přenositelnost mezi

Windows a Linux, kvalitní podporu MVVM, pohodlnou práci s CSV a SQLite a spolehlivou tvorbu PDF.

1.4.1 Jazyk C#

C# je moderní a obecně použitelný jazyk se zaměřením na produktivitu i výkon. Patří do ekosystému .NET, který nabízí bohaté nástroje a knihovny. Podle oficiální dokumentace je C# multiplatformní a používá se napříč oblastmi .NET (desktop, web, cloud); důraz klade na čitelný a bezpečný kód [1]. .NET je zdarma, otevřený a běží na Windows, Linuxu i macOS, takže umožňuje vyvíjet jeden kód pro více platforem [2].

Pro uživatelské rozhraní používám Avalonia UI, které přirozeně vychází z MVVM (Model–View–ViewModel) a datových vazeb. Dokumentace popisuje oddělení View a ViewModelu přes binding a DI, což vede k čistému návrhu a snadno testovatelné prezentační logice [3]. Reaktivní stav a příkazy zajišťuje ReactiveUI, které je cílené na MVVM a podporuje deklarativní práci se stavem [4].

Pro lokální úložiště volím SQLite. Používá standardní SQL, má kvalitní integraci do .NET a oficiální dokumentace přesně vymezuje podporovanou syntaxi a funkce [5]. To usnadňuje práci s tabulkami, agregacemi a filtry, které aplikace pro výkazy potřebuje.

Pro export do PDF používám knihovnu QuestPDF. Nabízí „fluent“ API v C# a je navržena pro deterministické rozvržení tabulek a textu v .NET aplikacích, jak uvádí domovská dokumentace.[6]

Existují i rozumné alternativy. TypeScript/JavaScript + Electron umožňuje psát desktop v JS/HTML/CSS a díky Chromiu a Node.js běží napříč platformami; cena za to jsou vyšší nároky na paměť a jiný model UI než XAML/MVVM [7]. Python + Qt (PySide6/Qt for Python) nabízí rychlé prototypování a multiplatformní GUI, ale pro tento projekt preferuji silnou typovou kontrolu C# a těsnou integraci s .NET ekosystémem [8][9]. Ze „světa JVM“ je zajímavá Kotlin Compose Multiplatform, která sdílí deklarativní UI napříč desktopem, Androidem, iOS a webem; mobilní cíle však nepotřebuji a zvolený stack C#/.NET lépe navazuje na použité knihovny a zkušenosti [10].

Shrnutí: kombinace C#/.NET + Avalonia UI + ReactiveUI tvoří ucelený, testovatelný a multiplatformní základ. SQLite pokrývá lokální databázi se srozumitelným SQL a QuestPDF spolehlivě řeší tiskové výstupy. Tato volba splňuje požadavky této práce: přenositelnost mezi Windows a Linux, podpora MVVM, pohodlná práce s CSV/SQLite a kvalitní generování PDF.

1.4.2 Jazyk SQL

Tato podkapitola shrnuje teorii potřebnou pro práci se SQL v kontextu lokální databáze SQLite a její integrace do aplikace v .NET. Cílem je vysvětlit základní pojmy a ukázat, jak je použít v praxi.

Data jsou uložena v tabulkách. Tabulka má sloupce a řádky. Každá tabulka by měla mít primární klíč (PK) pro jednoznačnou identifikaci řádku. Vztahy mezi tabulkami se vyjadřují pomocí cizích klíčů (FK). SQLite cizí klíče podporuje a je možné zapnout jejich kontrolu, aby se udržela referenční integrita.[5]

SQL se dělí na tři části. **DDL** (Data Definition Language) slouží k vytváření a změnám schématu, typicky příkazy *CREATE TABLE* a *ALTER TABLE*. **DML** (Data Manipulation Language) pracuje s obsahem tabulek, tedy *INSERT*, *UPDATE* a *DELETE*. **DQL** (Data Query Language) vrací data především pomocí *SELECT* s částmi jako *WHERE*, *ORDER BY* a *GROUP BY*.

Kvalitu dat hlídají omezení. Patří sem *NOT NULL*, *UNIQUE*, *CHECK* a *FOREIGN KEY*. Tato omezení brání chybám, jako jsou prázdné povinné hodnoty, duplicity nebo nelogické údaje (např. záporný počet hodin). SQLite přesně definuje význam i syntaxi těchto omezení.[5]

Změny se provádějí v transakcích. Používají se příkazy *BEGIN*, *COMMIT* a *ROLLBACK*. Cílem je splnit vlastnosti ACID: atomičnost, konzistenci, izolaci a trvalost. Díky tomu nedojde k poškození dat ani při částečně provedených úpravách. Dokumentace SQLite popisuje také režimy transakcí (deferred, immediate, exclusive).[5]

Pro přehledy a součty jsou důležité dotazy a výkon. Hodí se agregace *SUM*, *COUNT*, *AVG*, seskupení *GROUP BY* a filtry *WHERE*. SQLite podporuje CTE (dočasně pojmenované výsledky) včetně rekurzivních dotazů a také okenní funkce pro pokročilé agregace [5]. Výkon výrazně zlepšují indexy, například na datových sloupcích a klíčích. Indexy zrychlují vyhledávání i spojování tabulek.[5]

SQLite používá tzv. manifest typing. Interně rozlišuje pět tříd úložiště: *INTEGER*, *REAL*, *TEXT*, *BLOB* a *NULL*. Sloupce mají typovou afinitu (např. „INTEGER“ nebo „TEXT“). Tento přístup je flexibilní, ale vyžaduje pečlivé zacházení s formáty, hlavně u dat a časů, a konzistentní validaci v aplikaci.[5]

U datumu a času se běžně používá formát ISO-8601 jako text, případně UNIX epoch jako číslo. Pro české prostředí je důležité i collation, tedy způsob porovnávání řetězců. SQLite k těmto oblastem nabízí doporučené postupy a vestavěné funkce, například date a strftime.[5]

Bezpečnost dotazů je klíčová. Při práci s uživatelskými vstupy je nutné používat parametrizované dotazy nebo ORM. Skládání SQL z řetězců je rizikové a může vést k SQL injection. V .NET se používají připravené parametry nebo ORM vrstva.

Integrace do .NET je přímočará. Lze pracovat přímo přes ADO.NET nebo použít Entity Framework Core (EF Core), pro který existuje oficiální provider pro SQLite.[21]

Shrnutí. SQL/SQLite poskytuje stabilní a přenositelné úložiště pro výkazy a přehledy. Omezení a transakce chrání integritu dat. Indexy a agregace umožňují rychlé výpočty součtů. V kombinaci s .NET (a případně EF Core) je práce s daty jednoduchá a bezpečná.[5][21]

1.5 Framework Avalonia UI

Tato kapitola shrnuje základy Avalonia UI, otevřeného frameworku pro .NET. Zaměřuje se na pojmy, které jsou důležité pro návrh, implementaci a testování aplikace s architekturou MVVM (Model–View–ViewModel).

Avalonia je XAML-like framework pro deklarativní tvorbu uživatelských rozhraní. Umožňuje oddělit vzhled od logiky a přirozeně podporuje MVVM. Dokumentace jej doporučuje jako hlavní vzor a popisuje propojení View a ViewModelu pomocí datových vazeb a příkazů.[3]

Uživatelské rozhraní je zapsané v souborech AXAML. Tyto View neobsahují aplikační logiku; ta je ve třídách ViewModel, které vystavují vlastnosti a příkazy. Model tvoří doménová data a služby. Tato separace snižuje provázanost a zlepšuje testovatelnost.[3]

Vazby (binding) propojují vlastnosti ViewModelů s prvky UI. Příkazy (command) spouštějí akce z UI bez psaní kódu do View (code-behind). Díky tomu zůstává prezentace deklarativní a ubývá „lepidlového“ kódu.[3]

Pro reaktivní stav a příkazy používám ReactiveUI. Poskytuje reaktivní vlastnosti, příkazy a pozorovatelné toky dat. ViewModely lze díky tomu testovat bez běžícího UI, a přitom zachovat čisté MVVM.[4][12]

Koncept **Behaviors** (známý z WPF) umožňuje přidat opakovatelné interakce k prvkům UI bez kódu ve View. Microsoft jej popisuje pro WPF [14]. V Avalonia UI je dostupná knihovna **Avalonia.Xaml.Behaviors** [15] a dokumentace s příklady (např. spouštění příkazů na určité

události, podmíněné chování) [16]. Behaviors pomáhají udržet View jednoduché a znovupoužitelné.

Stejně jako jiné XAML frameworky podporuje Avalonia styly, šablony ovládacích prvků (templates) a sdílené zdroje (resources). Umožňují upravit vzhled a držet konzistenci bez zásahu do logiky, což dobře zapadá do MVVM: vzhled se mění ve View, chování zůstává ve ViewModelu.[3]

Avalonia UI přináší deklarativní GUI, čisté oddělení vzhledu a logiky (MVVM), možnost reaktivní implementace s ReactiveUI a rozšiřitelnost přes Behaviors. Tyto vlastnosti jsou pro tuto práci klíčové: usnadňují návrh náhledu tabulek EPD, validací a stavové logiky a zároveň udržují kód přehledný a testovatelný.

1.6 Vývojové prostředí

Tato kapitola popisuje nástroje použité při vývoji a důvody jejich volby. Cílem je, aby šlo projekt snadno spustit i na jiném počítači a v budoucnu. Pro vývoj v C#/.NET používám IDE a pro práci s lokální databází SQLite samostatný nástroj

1.6.1 Microsoft Visual Studio

Microsoft Visual Studio je hlavní IDE pro tento projekt. Umožňuje psát kód, spravovat závislosti přes NuGet, ladit a spouštět aplikaci. Pomáhá IntelliSense (nápověda a doplňování), refaktoring, vestavěný debugger a profilace výkonu.

IDE má integrovanou podporu Git/GitHub a Azure DevOps, takže lze sledovat změny, větvit a slučovat kód přímo v prostředí. Pro týmovou práci je k dispozici i Live Share.

Pro návrh uživatelského rozhraní používám Avalonia (AXAML) s oficiálním rozšířením pro VS; balíčky se instalují přes NuGet. Poznámka k platformám: na Windows běží Visual Studio. Na Linuxu lze projekt otevřít ve Visual Studio Code nebo používat .NET CLI se stejným zdrojovým kódem.

1.6.2 SQLiteStudio

SQLiteStudio je grafický nástroj pro práci se soubory databáze SQLite. Umožňuje prohlížet a měnit tabulky, indexy a záznamy, spouštět SQL dotazy se zvýrazněním syntaxe a využívat historii dotazů. Podporuje import/export (např. CSV, SQL dump) a umí zobrazit vztahy mezi tabulkami (ER náhled). V praxi jej používám k rychlé kontrole schématu, přípravě testovacích dat a ladění dotazů mimo běžící aplikaci.

1.7 Knihovny

Knihovny představují znovupoužitelné softwarové komponenty, které rozšiřují funkcionalitu aplikací bez nutnosti implementace všech částí kódu od začátku. Mohou obsahovat hotové funkce, třídy či šablony pro řešení specifických úloh.

V prostředí .NET se pro správu knihoven využívá systém NuGet, jenž umožňuje snadnou instalaci, aktualizaci a odstraňování závislostí. Použití knihoven zrychluje vývoj, snižuje chybovost a zajišťuje vyšší kvalitu softwaru, protože vývojáři pracují s prověřenými a otestovanými komponentami.

2 PRAKTICKÁ ČÁST

Tato kapitola popisuje realizaci aplikace pro evidenci a vyhodnocení pracovní doby vyučujících. Vysvětluje volbu jazyků a frameworku, použitou architekturu a vzor MVVM. Dále rozebírá klíčové části implementace: práci s databází, import rozvrhu, generování výstupů a uživatelské rozhraní.

2.1 Programovací jazyky

V projektu kombinuji C# (.NET) pro aplikační logiku a AXAML pro deklarativní popis uživatelského rozhraní. C# řeší práci s daty, algoritmy a integrace (import CSV, přístup do SQLite, generování PDF). AXAML popisuje obrazovky a vazby na MVVM. Tato kombinace udržuje čisté oddělení prezentace a logiky a zároveň umožňuje těsnou integraci přes data binding a příkazy.

2.1.1 Jazyk C#

C# tvoří páteř řešení. Modely mapované přes Entity Framework Core na SQLite. Schéma v kódu odpovídá schématu databáze.

Events – jednotná tabulka pro všechny denní události (výuka, práce a jiné dny pracovního klidu, dovolená, nemoc, ošetřování člena rodiny, návštěva lékaře, služební cesta).

Časové údaje:

StartTime, EndTime, AllDay

Typ a původ:

EventType, IsAutoGenerated, ParentEventId, ImportBatchId, ImportLabel.

Příchody a obědy:

ArrivalTime, DepartureTime, LunchStart, LunchEnd.

Kvalita dat:

HasCollision (označení kolize – časový překryv dvou událostí ve stejném čase).

GlobalSettings – výchozí pravidla a limity: hodnoty pro období, defaultní příchody/odchody a obědy po dnech, min./max. délka přestávky, názvy auto-událostí před/po obědě.

UserSettings – uživatelská přenastavení pro konkrétní datum (příchod, odchod, začátek a konec oběda).

EFMigrationsHistory – historie migrací schématu.

Služby:

Import rozvrhu (CSV): načtení šarže, určení typu událostí, uložení do Events

Vyrovňávání a validace hodin: aplikace pravidel EPD, generování podpůrných bloků přestávky, jemné posuny v povolených mezích, označení kolizí.

Export do PDF (QuestPDF): měsíční tabulka s hlavičkami, součty, zvýrazněním svátků a celodenních stavů.

ViewModely: filtrace rozvrhu, náhled/validace, spouštění importu a exportu; napojení na AXAML přes vazby a příkazy.

Aplikace používá `async/await` pro čtení souborů, práci s databází a generování PDF, aby zůstalo GUI responzivní. LINQ slouží k projekcím, filtrům a agregacím (týdenní a měsíční součty). Při ukládání a úpravách probíhá kontrola: bloky v jednom dni se nesmí nelogicky překrývat; celodenní události se zapisují jedním záznamem (*AllDay* = true); přestávky respektují nastavené meze; týdenní/měsíční součty odpovídají požadavkům

Proč C#: silné typování odhalí mnoho chyb už při kompilaci, knihovny .NET zrychlují vývoj a jednotné úložiště Events zjednodušuje dotazování i generování výstupů. Rozdělení na modely–služby–ViewModely udržuje kód přehledný a připravený na rozšíření.

2.1.2 Jazyk XAML

AXAML deklarativně popisuje obrazovky v Avalonia UI: průvodce importem CSV, kalendářový přehled, editor denních záznamů a náhled před exportem PDF. View neobsahuje aplikační logiku; ta je ve ViewModelech v C#.

Vazby propojují prvky rozhraní s vlastnostmi ViewModelu (např. kolekce událostí měsíce, agregované součty). Uživatelské akce spouštějí příkazy (*GenerateEPDCommand*, *CreateEventCommand*, *ShowPdfPreview*). Změny stavu se okamžitě projeví v UI bez ručního překreslování.

Opakující se části (řádky/denní buňky, značky svátků a absencí) jsou definovány pomocí *DataTemplate* a *ControlTemplate*. Styly sjednocují barvy, písma a odsazení, takže lze měnit vzhled bez zásahu do C#.

Ve spojení s **ReactiveUI** jsou ViewModely reaktivní: změny dat (např. vložení půldenní dovolené) okamžitě přepočítají týdenní součet a aktualizují náhled tabulky. Tím zůstává rozhraní plynulé i při práci s většími měsíci.

AXAML drží prezentační vrstvu izolovanou, takže se „co se zobrazuje“ (AXAML) a „co se počítá“ (C#) může vyvíjet nezávisle. Zároveň zůstává zachována jednoduchá integrace přes bindingy a příkazy, což zrychluje iterace nad UI i logikou.

2.2 Struktura aplikace

Aplikace vychází ze vzoru MVVM, který je pro Avalonia UI přirozený. Vzor odděluje prezentační vrstvu od logiky a dává jasný rámec pro organizaci kódu. Díky tomu lze řešení snadno rozšiřovat, testovat a udržovat, aniž by se míchaly části, které mají zůstat nezávislé.

Model tvoří perzistentní entity mapované pomocí Entity Framework Core na databázi SQLite. Klíčové třídy jsou *Event*, *UserSettings* a *GlobalSettings*.

Event popisuje konkrétní události dne (výuka, práce, dovolená, nemoc ap.). Uchovává časy začátku a konce, typ události a související údaje pro příchod/odchod a oběd.

UserSettings drží individuální nastavení pro vybraný den (příchod, odchod, začátek a konec oběda).

GlobalSettings definuje výchozí pracovní časy, parametry přestávek, informace o vyučujícím a režimy období.

O přístup k databázi se stará *AppDbContext*, který kromě definice tabulek řeší i uložení časových údajů a mapování vztahů mezi entitami.

View představují .axaml soubory, které určují vzhled jednotlivých částí rozhraní. Patří sem například *MainWindow.axaml* s navigací a kalendářem, *DayView.axaml*, *WeekView.axaml* a *MonthView.axaml* pro různé pohledy na rozvrh, dále dialogy jako *CreateEventDialog.axaml* nebo *GlobalUserSettingsView.axaml*. Součástí pohledů jsou i specifická chování (Behaviors), která přidávají interaktivitu – například *ShowCollisionMessageBehavior* zajišťuje zobrazení upozornění na kolize událostí. Díky těmto chováním zůstává kód čistý a opakovaně použitelný. Uživatelské rozhraní je navrženo tak, aby bylo intuitivní: levý panel hlavního okna nabízí přepínání mezi denním, týdenním a měsíčním pohledem, správu nastavení a generování PDF, zatímco pravý panel zobrazuje samotná data kalendáře.

ViewModel vrstva funguje jako prostředník mezi modelem a pohledem. Hlavní třídou je *MainWindowViewModel*, který koordinuje načítání dat a přepínání mezi jednotlivými režimy zobrazení. Samostatné ViewModely pro den, týden a měsíc (*DayViewModel*, *WeekViewModel*,

MonthViewModel) zajišťují logiku a zpracování dat specifické pro danou kategorii kalendáře. Pro práci s globálním nastavením existuje *GlobalUserSettingsViewModel*, který umožňuje načítat a ukládat pracovní intervaly, parametry přestávek a další údaje. Důležitou roli hraje také *CreateEventDialogViewModel*, jenž se stará o validaci vstupů při vytváření a editaci událostí, a *PdfPreviewViewModel*, který připravuje data pro generování a vyvážení pracovních hodin v oficiálním PDF. Veškeré změny dat se do rozhraní propisují pomocí mechanismu data bindingu, což znamená, že aktualizace hodnot ve ViewModelu se okamžitě projeví v odpovídajícím pohledu a uživatel vždy pracuje s aktuálními informacemi.

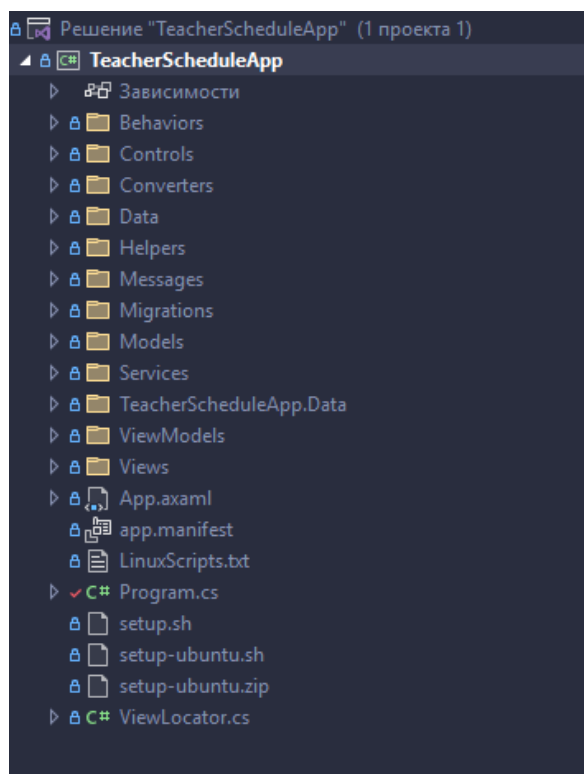
Servisní vrstva doplňuje architekturu o centralizovanou logiku, která by neměla být přímo součástí ViewModelů. Klíčovým prvkem je *EventService*, který kromě klasických CRUD operací implementuje také algoritmus pro vyvažování odpracovaných hodin mezi jednotlivými dny. Tento mechanismus automaticky vyrovnává přebytky a nedostatky v pracovním fondu tak, aby měsíční bilance odpovídala nastaveným pravidlům, aniž by uživatel musel data upravovat ručně. *WorkingHoursCalculatorService* zajišťuje výpočty odpracovaného času, *AutomaticEventsGeneratorService* vytváří výchozí události na základě globálních nastavení a rozvrhu, a *PdfService* připravuje výsledné výstupy ve formátu PDF. Zvláštní roli hraje komponenta *EPDGenerator*, která je určena pro načítání rozvrhu vyučujícího ze souboru CSV (exportovaného z IS STAG) a pro převod těchto dat na interní události aplikace. Díky tomu může uživatel rychle naplnit svůj kalendář skutečným rozvrhem a následně využít ostatní nástroje aplikace pro vyvažování, úpravy a export do oficiálních dokumentů. Dále *GlobalSettingsService* a *SettingsService* poskytují rozhraní pro perzistenci uživatelských i globálních konfigurací.

Díky této architektuře je aplikace přehledná, modulární a snadno rozšiřitelná. Každá vrstva má jasně definovanou zodpovědnost: Model obstarává data a jejich uložení, View definuje vzhled a chování uživatelského rozhraní, ViewModel zprostředkovává interakci mezi uživatelem a logikou, a Services centralizují výpočty a komplexní operace. Tento přístup umožnil realizovat robustní desktopovou aplikaci, která je schopná automaticky generovat události, vyrovnávat pracovní fond, zajišťovat multiplatformní běh a poskytovat uživateli komfortní nástroje pro evidenci pracovní doby.

2.4 Souborová struktura aplikace

Struktura projektu je navržena tak, aby jasně oddělovala jednotlivé vrstvy a funkce aplikace a zároveň podporovala zásady přehlednosti, udržitelnosti a rozšiřitelnosti. Díky tomuto

členění lze snadno vyhledat části kódu související s databází, prezentační logikou, uživatelským rozhraním či podpůrnými funkcemi. Projekt je rozdělen do samostatných složek, z nichž každá se zaměřuje na konkrétní oblast.



Obrázek 3: Souborová struktura aplikace

Celkově tato souborová struktura poskytuje jasné oddělení zodpovědností, usnadňuje orientaci v projektu a umožňuje efektivní vývoj i budoucí rozšiřování. Každá složka má svou specifickou úlohu, díky čemuž je aplikace modulární a přehledná, a to jak pro jednoho vývojáře, tak i pro týmovou spolupráci.

2.4.1 Složka Behaviors

Složka Behaviors obsahuje třídy využívající Avalonia.Xaml.Behaviors. Jejich cílem je přidat specifické interakce k uživatelským prvkům, aniž by bylo nutné měnit jejich šablony. Nachází se zde například mechanismy pro zachytávání kliknutí na buňky kalendáře či události, komponenty pro zavírání dialogů a potvrzování jejich akcí nebo chování zajišťující zobrazování chybových a kolizních hlášení. Díky tomuto přístupu zůstává kód rozhraní čistý a interakce lze snadno měnit.

2.4.2 Složka Controls

Další částí je složka Controls, která sdružuje vlastní uživatelské prvky rozšiřující základní komponenty Avalonia UI. Tyto prvky slouží k zobrazení událostí v kalendáři nebo k rozvržení panelů a díky své znovupoužitelnosti přispívají k modulárnosti a konzistenci aplikace.

2.4.3 Složka Converters

Transformace dat mezi ViewModely a zobrazením je soustředěna ve složce Converters. Převodníky implementující `IValueConverter` zde slouží k převodu logických hodnot na viditelnost či barvy, k výpočtu pozic událostí v kalendáři nebo k transformaci časových údajů na grafické rozměry. Tento přístup umožňuje udržet čistý MVVM vzor a logiku transformace oddělit od prezentace.

2.4.4 Složka Data

Datovou vrstvu představuje složka Data obsahující kontext `AppDbContext`. Ten definuje přístup k databázi prostřednictvím Entity Framework Core, deklaruje sady entit pro události a nastavení a zajišťuje připojení k databázovému souboru SQLite uloženému v uživatelském adresáři. Současně mapuje vlastnosti na databázové sloupce a obsahuje konvertory pro jednotné ukládání časových hodnot.

```

Ссылка: 42 | Egor Razboinikov, 3 дн. назад | Автор: 1, изменений: 3
public class AppDbContext : DbContext
{
    Ссылка: 45 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    public DbSet<Event> Events { get; set; }
    Ссылка: 6 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    public DbSet<UserSettings> UserSettings { get; set; }
    Ссылка: 4 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    public DbSet<GlobalSettings> GlobalSettings { get; set; }

    Ссылка: 0 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        string baseDir;
        if (RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
            baseDir = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
        else
            baseDir = Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.UserProfile), ".config");

        var appDir = Path.Combine(baseDir, "TeacherScheduleApp");
        Directory.CreateDirectory(appDir);

        var dbPath = Path.Combine(appDir, "teacherapp.db");
        optionsBuilder.UseSqlite($"Data Source={dbPath}");
    }

    Ссылка: 0 | Egor Razboinikov, 3 дн. назад | Автор: 1, изменений: 3
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        var dateTimeConverter = new ValueConverter<DateTime, string>(
            v => v.ToString("yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture),
            v => DateTime.ParseExact(v, "yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture)
        );
        modelBuilder.Entity<Event>(b =>
        {
            b.HasKey(e => e.Id);
            b.Property(e => e.Id).ValueGeneratedOnAdd();
        });
        modelBuilder.Entity<Event>()
            .HasOne<Event>()
            .WithMany()
            .HasForeignKey(e => e.ParentEventId)
            .IsRequired(false)
            .OnDelete(DeleteBehavior.Cascade);

        modelBuilder.Entity<Event>()
            .Property(e => e.StartTime)
            .HasConversion(dateTimeConverter);

        modelBuilder.Entity<Event>()
            .Property(e => e.EndTime)
            .HasConversion(dateTimeConverter);
        base.OnModelCreating(modelBuilder);
    }
}

```

Obrázek 4 Ukázka kódu třídy AppDbContext

2.4.5 Složka Helpers

Podpůrnou funkci plní složka Helpers, kde jsou umístěny univerzální třídy využívané napříč aplikací. Patří sem například logika pro získání reference na hlavní okno nebo komponenta zpracovávající seznam státních svátků prostřednictvím knihovny Nager.Date.

2.4.6 Složka Messages

Komunikaci mezi částmi systému zajišťuje složka Messages. Obsahuje definice zpráv, které informují ViewModely o důležitých změnách, například o vytvoření nových automatických událostí, dokončení generování PDF či úpravě nastavení. Díky tomuto mechanismu spolu moduly spolupracují bez přímých závislostí.

2.4.7 Složka Models

Datové entity jsou definovány ve složce Models. Nachází se zde třídy popisující události, globální i uživatelská nastavení nebo jednotlivé bloky výuky. Jedná se o jednoduché objekty mapované prostřednictvím Entity Frameworku na databázové tabulky.

2.4.8 Složka Services

Složka Services je jádrem obchodní logiky. Obsahuje služby starající se o správu událostí, generování automatických bloků, výpočet pracovních hodin nebo tvorbu PDF souborů. Zvláštní význam má algoritmus vyvažování pracovní doby, který se snaží rozdělit odpracované hodiny tak, aby se měsíční balance co nejvíce blížila očekávané normě. Do této části patří také třída EPDGenerator, která čte rozvrh učitele ze souboru CSV a převádí jej na databázové události.

2.4.9 Složka ViewModels

Prezentační logiku soustřeďuje složka ViewModels. Každý ViewModel propojuje data s konkrétním uživatelským rozhraním a definuje příkazy pro práci s aplikací. Hlavní ViewModel zajišťuje přepínání mezi denním, týdenním a měsíčním režimem, další spravují dialogy, nastavení nebo náhled PDF.

2.4.10 Složka Views

Vizuální vrstvu tvoří složka Views, v níž se nacházejí šablony .axaml a jejich code-behind. Tyto soubory definují vzhled hlavního okna, jednotlivých kalendářních pohledů a dialogů. Jejich oddělení od logiky usnadňuje úpravy uživatelského rozhraní.

2.4.11 Složka Kořenové soubory projektu

Kořenové soubory projektu obsahují deklarace globálních zdrojů a startovací logiku. App.axaml definuje styly a šablony, jeho code-behind inicializuje hlavní okno a životní cyklus aplikace. Manifest určuje metadata pro Windows, zatímco Program.cs představuje vstupní bod aplikace. ViewLocator pak zajišťuje propojení ViewModelů s odpovídajícími Views. Celkově je struktura navržena tak, aby byla modulární, přehledná a snadno rozšiřitelná.

2.5 Struktura databáze

Aplikace používá SQLite jako lokální úložiště v jednom souboru a Entity Framework Core (EF Core) pro mapování entit a migrace. Schéma je záměrně jednoduché a neobsahuje relační vazby na úrovni databáze (tj. nejsou definovány cizí klíče ani referenční omezení).

2.5.1 Přehled tabulek

Events – jednotná tabulka pro všechny události dne, které zapisuje uživatel nebo je vytvoří aplikace Neobsahuje žádné systémové záznamy pro víkendy nebo svátky:

Id – primární klíč (PK).

StartTime, EndTime – začátek/konec bloku (časové úseky dne).

EventType – typ události (práce, D, N, OČR, L, PC, S).

Title, Description – textové popisky.

AllDay – je-li záznam celodenní (např. dovolená na celý den).

ArrivalTime, DepartureTime, LunchStart, LunchEnd – reálné hrany dne a polední přestávka pro daný den (pokud je relevantní).

IsAutoGenerated – automaticky vložený blok (např. přestávka dopočítaná algoritmem).

ParentEventId – identifikace zdrojové události pro blok (logická reference; bez cizího klíče).

HasCollision – příznak zjištěného překryvu dvou stavů.

ImportBatchId, ImportLabel – původ dat (šarže importu).

AutoGeneratedForDate – datum, pro které byl auto-blok vytvořen.

IsDeleted – měkké smazání (záznam zůstává v historii, ale běžné dotazy ho neberou v potaz).

GlobalSettings – tabulka globálních pravidel a výchozích hodnot (pro rok/semestr):

identifikace vyučujícího (jméno, útvar),

výchozí okno dne (globální příchod/odchod),

denní defaulty (pondělí–pátek: příchod, odchod, oběd *LunchStart/LunchEnd*),

limity přestávky (*MinBreakDuration, MaxBreakDuration*),

názvy automatických bloků (před/po obědě).

UserSettings – tabulka uživatelských přenastavení pro konkrétní kalendářní dny:

Id (PK), *Date* (den),

ArrivalTime, DepartureTime, LunchStart, LunchEnd.

Existuje-li záznam pro daný den, přebíje výchozí hodnoty z *GlobalSettings*.

_EFMigrationsHistory – technická tabulka EF Core s historií migrací schématu.

2.5.2 Bez cizích klíčů: záměr a důvody

Relační vazby nejsou uplatněny na úrovni databáze (SQLite foreign_keys nejsou vyžadovány).

Důvody:

Jednoduchost a přenositelnost souboru.

SQLite je souborová databáze. Udržovat ji „štíhlou“ usnadňuje zálohování, kopírování mezi stroji a případné ruční zásahy při testování.

Tolerantní import po šaržích.

Import z CSV probíhá po dávkách (*ImportBatchId*). Dočasně mohou vzniknout neúplné či „čekající“ záznamy (např. auto-bloky přidáné až po validaci). Absence cizích klíčů zjednodušuje do-import a následné dorovnání bez nutnosti pořadí insertů a bez selhání transakce kvůli referencím.

Logika a pravidla jsou v aplikaci

Všechny invarianty (překryvy, priority události, korektní měsíční/týdenní součty) i „logické vazby“ (např. *ParentEventId*) hlídá servisní vrstva a validace při ukládání. Pravidla EPD se tak dají verzovat a měnit v kódu, nikoli v triggerch databáze.

Výkon a jednoduché dotazy.

Typické dotazy se opírají o *StartTime* a filtrování období; nepotřebujeme joiny přes FK. Indexy nad časem a šaržemi dávají větší reálný přínos než formální referenční omezení.

2.5.3 Shrnutí rozhodnutí

Žádné cizí klíče a bez uložených víkendů/svátků menší, odolnější soubor DB, jednodušší importy, snadné vracení šarží a levnější změny pravidel.

Aplikační logika nese odpovědnost za integritu: validace při importu/úpravách, jednotná mřížka času, pravidla EPD, označování kolizí.

Jednotná tabulka Events zjednodušuje dotazování a export, GlobalSettings a UserSettings pokrývají „výchozí vs. výjimky“ bez nutnosti relačních vazeb.

2.6 Uživatelské rozhraní aplikace

Uživatelské rozhraní aplikace je někdy nejdůležitějším aspektem, který určuje, jak snadná a intuitivní bude pro uživatele interakce s aplikací. Jednoduchý a přímočarý návrh uživatelského rozhraní zajišťuje, že aplikace je intuitivní, snadno se používá. To pomáhá uživateli rychle a snadno pochopit, jak aplikace funguje, a může efektivně využívat její funkce. Význam uživatelského rozhraní by se neměl podceňovat. Koneckonců je to tvář aplikace a měla by být atraktivní. Pokud rozhraní obsahuje velké množství prvků nebo je obtížné se v něm orientovat, může to mít negativní dopad na uživatelský zážitek. Naopak dobře navržené uživatelské rozhraní s jasným vizuálem, a nejen strukturou může pro uživatele hrát velmi velkou roli. Klíčovými prvky dobrého uživatelského rozhraní jsou jednoduchost, přehlednost a konzistence.

2.6.1 Hlavní stránka aplikace

První věc, kterou uživatel uvidí po spuštění aplikace, je její hlavní stránka. Při prvním spuštění aplikace se také automaticky vytvoří události na celý rok. Hlavní stránka je rozdělena na dva hlavní panely. Levý panel slouží k různým interakcím s aplikací. Druhý panel slouží k zobrazení pohledů na daný den, týden nebo měsíc v závislosti na tom, co uživatel vybral v rozevíracím seznamu na levém panelu. Výchozí zobrazení je aktuální den.

Popíšeme podrobně levý panel. V horní části tohoto panelu jsou čtyři tlačítka, která slouží k vyvolání různých funkcí. První tlačítko „Vytvořit událost“ slouží k vyvolání dialogového okna (jeho popis je uveden níže). Druhé tlačítko „Generovat EPD“ slouží k vyvolání dialogového okna pro načtení souboru .csv s rozvrhem učitele ze STAG. Po výběru souboru se spustí proces vytváření událostí ze souboru. Třetí tlačítko „Globální nastavení“ slouží k otevření okna globálních nastavení (podrobný popis níže). Čtvrté tlačítko „Příklad PDF“ slouží k otevření dialogového okna pro prohlížení dokumentu ve formátu PDF. Za tlačítky se nachází kalendář s možností výběru dne, měsíce a roku. Pokud v tomto kalendáři kliknete na jakýkoli den, aplikace přejde a zobrazí údaje pro tento den. Dále je zde rozevírací prvek, který zobrazuje informace o nahraných souborech. Za ním se nachází rozevírací seznam, který slouží k výběru možnosti zobrazení dne, týdne nebo měsíce. Za seznamem následuje blok nastavení pro konkrétní den. Tam se nacházejí 4 hlavní pole, která slouží k zadání času příchodu, času odchodu a začátku a konce přestávky, a tlačítko, které uloží zadané informace. Stojí za zmínku, že tento blok je viditelný pouze v denním režimu. Za tímto blokem se zobrazují informace o tom, kolik hodin bylo odpracováno z normy.

Vytvořit událost

Načíst rozvrh

Globální nastavení

EPD náhled

srpen 2025

^

v

P	Ú	S	Č	P	S	N
28	29	30	31	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31
1	2	3	4	5	6	7

Načtený soubory

▼

Rozvrh

▼

Nastavení pro den:

25.08.2025

Příchod:

08:30

Odchod:

17:00

Oběd od:

12:30

Oběd do:

13:00

Uložit nastavení

Obrázek 5 Levý panel aplikace

← Den: 25 srp 2025 → Dnes	
04:00	
05:00	
06:00	
07:00	
08:00	● Dopolední pracovní doba
09:00	
10:00	
11:00	
12:00	
13:00	● Oběd ● Odpolední pracovní doba
14:00	
15:00	
16:00	
17:00	
18:00	

Obrázek 6 Pravý panel aplikace

2.6.2 Dialogové okno pro vytvoření události

Dialogové okno obsahuje následující prvky. První pole slouží k zadání nebo úpravě názvu události. Toto pole je povinné, při pokusu o vytvoření události bez názvu se zobrazí dialogové okno s chybou. Druhé pole již není povinné a slouží k přidání popisu události. Poté následuje tlačítko, které určuje, zda bude událost trvat celý den. Poté následuje blok pro nastavení data a času události. Za blokem pro nastavení data je prvek, který umožňuje vybrat typ události (pracovní, volno, služební cesta atd.). A posledními prvky jsou tlačítko pro zrušení, které zavře okno, a tlačítko pro uložení, které událost vytvoří a okno zavře.

Nová událost

Název

Popis

☐ Celý den

Začátek:

4 květen 2025 00 00

Konec:

4 květen 2025 00 00

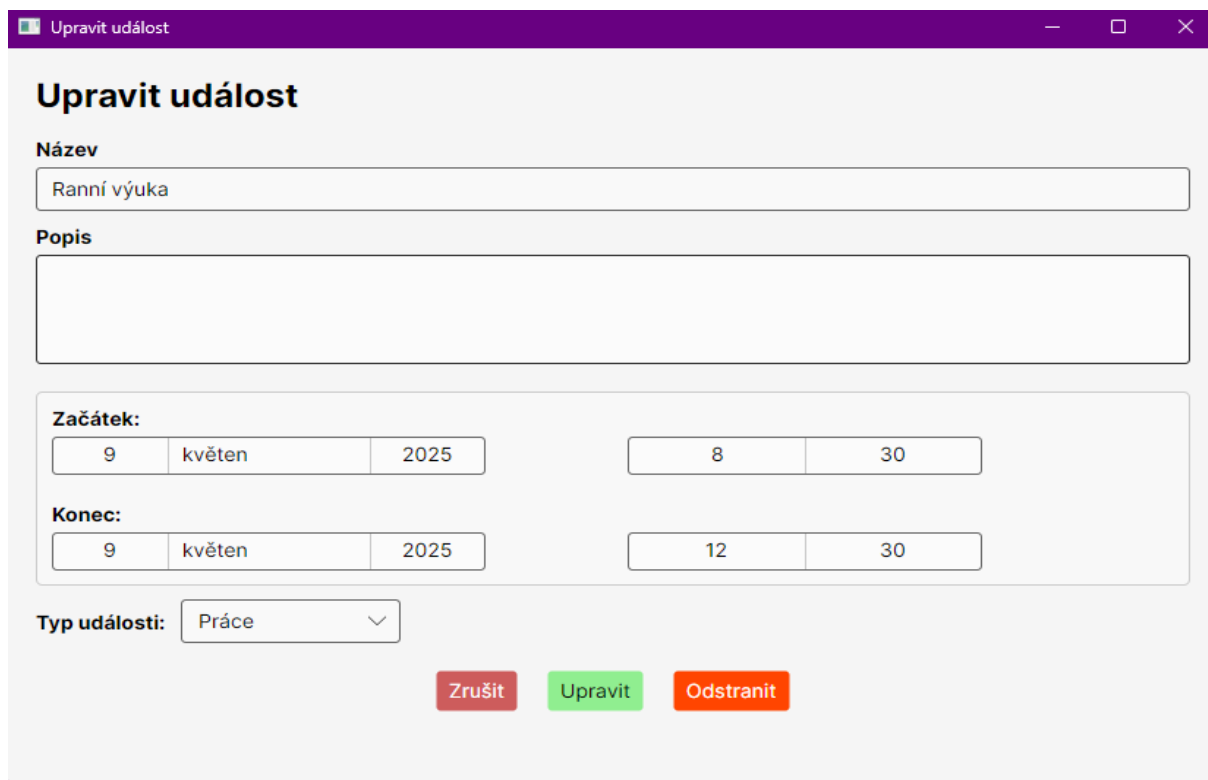
Typ události: Práce

Zrušit Vytvořit

Obrázek 7 Dialogové okno pro vytvoření

2.6.3 Dialogové okno pro změnu události

Dialogové okno pro změnu události obsahuje následující prvky. První pole slouží k zadání nebo změně názvu události. Toto pole je povinné, při pokusu o změnu události bez názvu se zobrazí dialogové okno s chybou. Druhé pole není povinné a slouží k přidání popisu události. Dále následuje blok pro nastavení data a času události. Za blokem pro nastavení data se nachází prvek, který umožňuje vybrat typ události (pracovní, volno, služební cesta atd.). Posledními prvky jsou tlačítka pro zrušení, které zavře okno, tlačítko pro uložení změn, které aktualizuje událost a zavře okno, a tlačítko, které slouží k odstranění události. Před odstraněním události bude uživatel požádán o potvrzení odstranění události.



Upravit událost

Název

Ranní výuka

Popis

Začátek:

9 květen 2025 8 30

Konec:

9 květen 2025 12 30

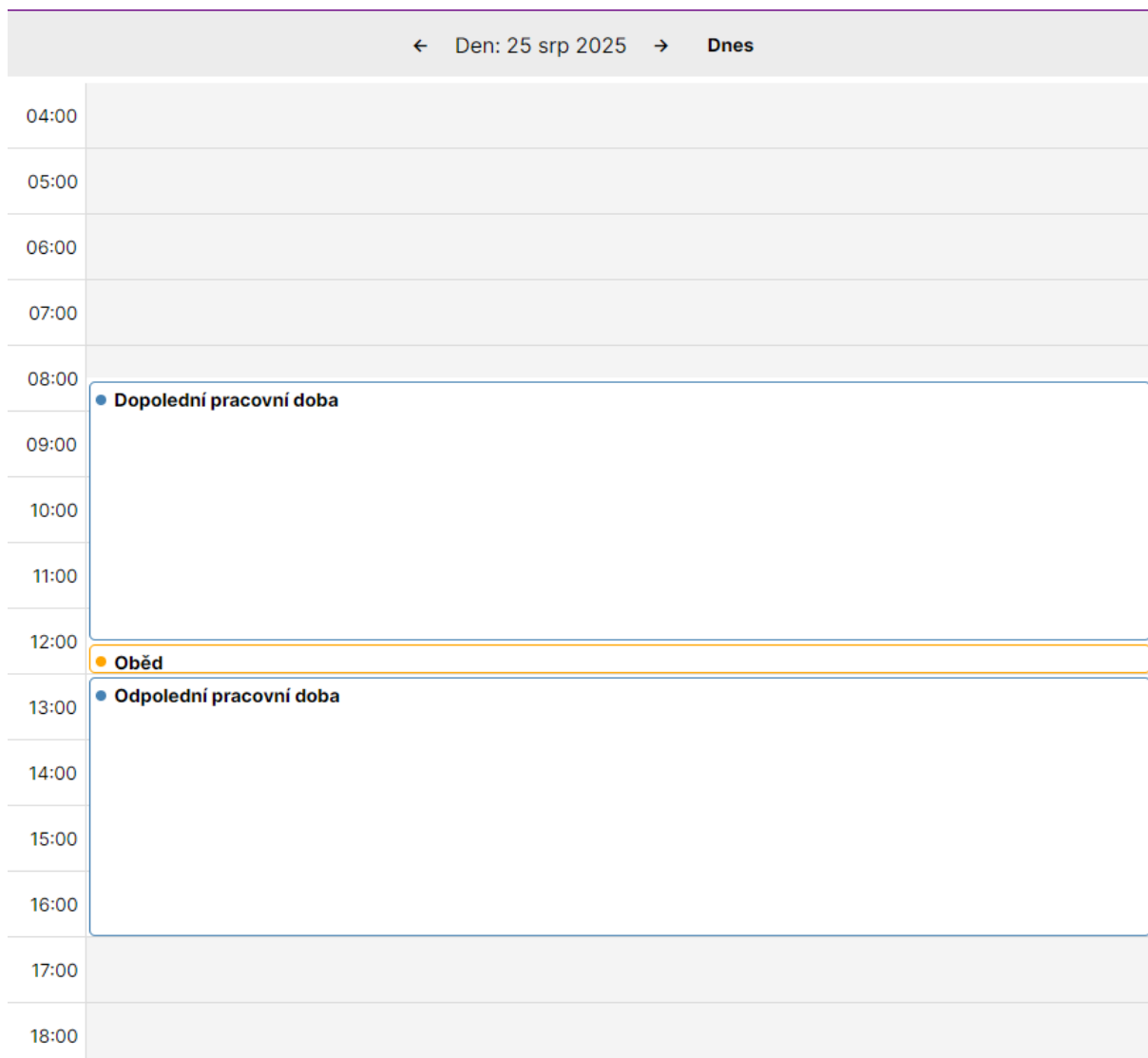
Typ události: Práce

Zrušit Upravit Odstranit

Obrázek 8 Dialogové okno pro změnu

2.6.4 Vizuální prezentace pro den

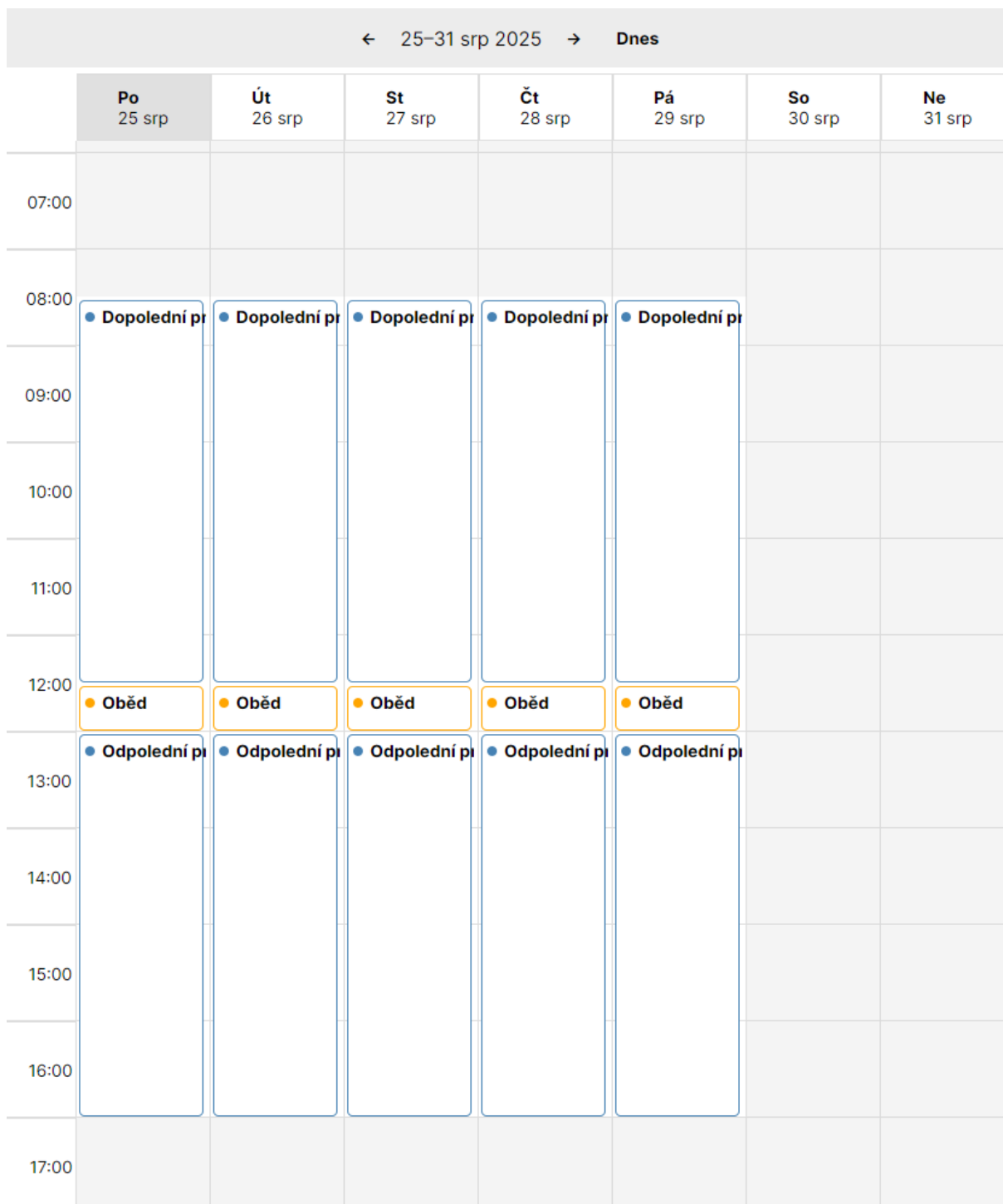
Vizuální prezentace pro daný den obsahuje mřížku s hodinami, pracovní interval je barevně odlišen od nepracovního. V horní části jsou tlačítka, která umožňují přepínat mezi dny. Tam se také nachází tlačítko, které přesune uživatele na aktuální den. Stejně tak zobrazuje události pro daný den, pokud nějaké existují.



Obrázek 9 Vizuální prezentace pro den

2.6.5 Vizuální prezentace pro týden

Vizuální prezentace pro týden obsahuje také tabulku s hodinami a dny, které tvoří týden. V horní části jsou tlačítka, která umožňují přepínat mezi týdny. Tam se také nachází tlačítko, které přesune uživatele na aktuální týden. Stejně tak zobrazuje události pro tento týden, pokud nějaké jsou.



Obrázek 10 Vizualní prezentace pro týden

2.6.6 Vizualní prezentace pro měsíc

Vizualní prezentace pro měsíc obsahuje také mřížku, ale již s dny, které spadají do tohoto měsíce. Státní svátky, víkendy a dny z předchozího nebo následujícího měsíce se liší barvou pozadí nebo barvou čísla dne. V horní části jsou tlačítka, která umožňují přepínat mezi měsíci. Tam se také nachází tlačítko, které přesune uživatele na aktuální měsíc. Stejně tak zobrazuje události pro tento měsíc, pokud nějaké jsou.

← srpen 2025 → Dnes						
Po	Út	St	Čt	Pá	So	Ne
28 ● 08:30 Dopol ● 12:30 Oběd	29 ● 08:30 Dopol ● 12:30 Oběd	30 ● 08:30 Dopol ● 12:30 Oběd	31 ● 08:30 Dopol ● 12:30 Oběd	1 ● 08:30 Dopol ● 12:30 Oběd	2	3
4 ● 08:30 Dopol ● 12:30 Oběd	5 ● 08:30 Dopol ● 12:30 Oběd	6 ● 08:30 oo	7 ● 08:30 Dopol ● 12:30 Oběd	8 ● 08:30 Dopol ● 12:30 Oběd	9	10
11 ● 08:30 Dopol ● 12:30 Oběd	12 ● 08:30 Dopol ● 12:30 Oběd	13 ● 08:30 Dopol ● 12:30 Oběd	14 ● 08:30 Dopol ● 12:30 Oběd	15 ● 08:30 Dopol ● 12:30 Oběd	16	17
18 ● 08:30 Dopol ● 12:30 Oběd	19 ● 08:30 Dopol ● 12:30 Oběd	20 ● 08:30 Dopol ● 12:30 Oběd	21 ● 08:30 Dopol ● 12:30 Oběd	22 ● 08:30 Dopol ● 12:30 Oběd	23	24
25 ● 08:30 Dopol ● 12:30 Oběd	26 ● 08:30 Dopol ● 12:30 Oběd	27 ● 08:30 Dopol ● 12:30 Oběd	28 ● 08:30 Dopol ● 12:30 Oběd	29 ● 08:30 Dopol ● 12:30 Oběd	30	31
1 ● 08:00 Dopol ● 12:00 Oběd	2 ● 08:00 Dopol ● 12:00 Oběd	3 ● 08:00 Dopol ● 12:00 Oběd	4 ● 08:00 Dopol ● 12:00 Oběd	5 ● 08:00 Dopol ● 12:00 Oběd	6	7

Obrázek 11 Vizualní prezentace pro měsíc

2.6.7 Globální nastavení

Globální nastavení obsahuje velké množství polí pro zadání informací o různých časových intervalech a dalších informací o uživateli, které budou použity pro vytváření automatických událostí, generování PDF a tak dále. Podrobný popis začneme tlačítky, která slouží k přepínání mezi zimním a letním semestrem, což uživateli umožňuje zadat různé údaje pro konkrétní semestr. Pole pro zadávání informací jsou stejná pro obě varianty. Dále následuje blok, který

odpovídá za globální interval pracovní doby. Poté následuje blok s možností zadání informací o příchodu do práce a odchodu z práce a také času přestávky. Tato pole jsou stejná pro všechny pracovní dny. Zložka další nastavení obsahuje pole pro zadání jména, katedry, které budou použity pro generování v PDF. Pole pro zadání omezení pro přestávku a také jména, která se používají pro vytváření automatických událostí. Na konci se také nachází tlačítko pro uložení.

[← Zpět ke kalendáři](#)**Rok 2025 · Letní semestr**Rok: 2025 ▼☐ Zimní ☒ Letní

Globální pracovní doba · Letní semestr
Od:
Do:

Pracovní doba · Letní semestr
Pondělí Úterý Středa Čtvrtek Pátek
Příchod:
Odchod:
Oběd, začátek:
Oběd, konec:

Další nastavení
Údaje pro PDF
Jméno:
Útvar:
Pauzy
Min. délka pauzy:
Max. délka pauzy:
Názvy událostí
Před obědem:
Pro oběd:
Po obědě:

Uložit nastavení

Obrázek 12 Globální nastavení

2.6.8 Ukázka PDF

Náhled PDF obsahuje dva prvky, se kterými lze interagovat. První slouží k přepínání mezi měsíci, druhý k uložení souboru do počítače. Také obsahuje obrázek, jak bude stažený soubor vypadat.

07-2025

Uložit PDF

Evidence pracovní doby, včetně přestávek v práci a práce přesčas

Fakulta elektrotechniky a informatiky
jméno: Radek Matoušek
útvár: Katedra informačních technologií

pracovní doba: 08:30–17:00 hod.
docházka za měsíc: červenec 2025

Fond prac. doby:
184 hodin

Den	Začátek pracovní doby	1. přestávka		2. přestávka		Konec pracovní doby	Odpracováno	Přesčas	Neodprac.	Poznámka)
		Od	Do	Od	Do					
1.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
2.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
3.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
4.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
5.										S
6.										S
7.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
8.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
9.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
10.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
11.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
12.										
13.										
14.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
15.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
16.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
17.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
18.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
19.										
20.										
21.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
22.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
23.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
24.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
25.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
26.										
27.										
28.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
29.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
30.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
31.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
Celkem							184:00:00	00:00:00	00:00:00	

kontr. č. Σ odprac. - přesčas + neodprac. hodin (odpovídá FPD): 184:00:00

podpis zaměstnance:

podpis nadřízeného pracovníka:

*) do poznámky: D – dovolená, N – nemoc, OČR – ošetřování, L – lékař, PC – pracovní cesta, S – svátek a ostatní dny pracovního klidu

Obrázek 13 Ukázka PDF

2.7 Algoritmy

Tato kapitola shrnuje implementované algoritmy, které zajišťují praktické fungování aplikace: jednak vyrovnávání hodin podle pravidel EPD jednak import rozvrhu ze souboru CSV exportovaného ze systému STAG. Popis odpovídá aktuálním třídám a metodám v projektu.

2.7.1 Algoritmus vyrovnávání hodin

Každý pracovní týden má dát 40 h (8 h/den) a zároveň musí sedět měsíční fond. Úpravy se dělají jen pomocí auto-bloků práce (automaticky generované úseky na okrajích dne). Manuální události se nepřesouvají. Kolize = časový překryv dvou událostí ve stejném čase — nesmí vznikat.

Příprava

1. Vstup: rok a měsíc + pravidla EPD (pracovní okno dne: příchod–odchod, délka oběda, svátky).

2. Pracovní dny: sestav dny měsíce bez víkendů a svátků.
3. Týdny: rozděl pracovní dny do ISO týdnů (stejně jako v kalendáři ISO 8601).

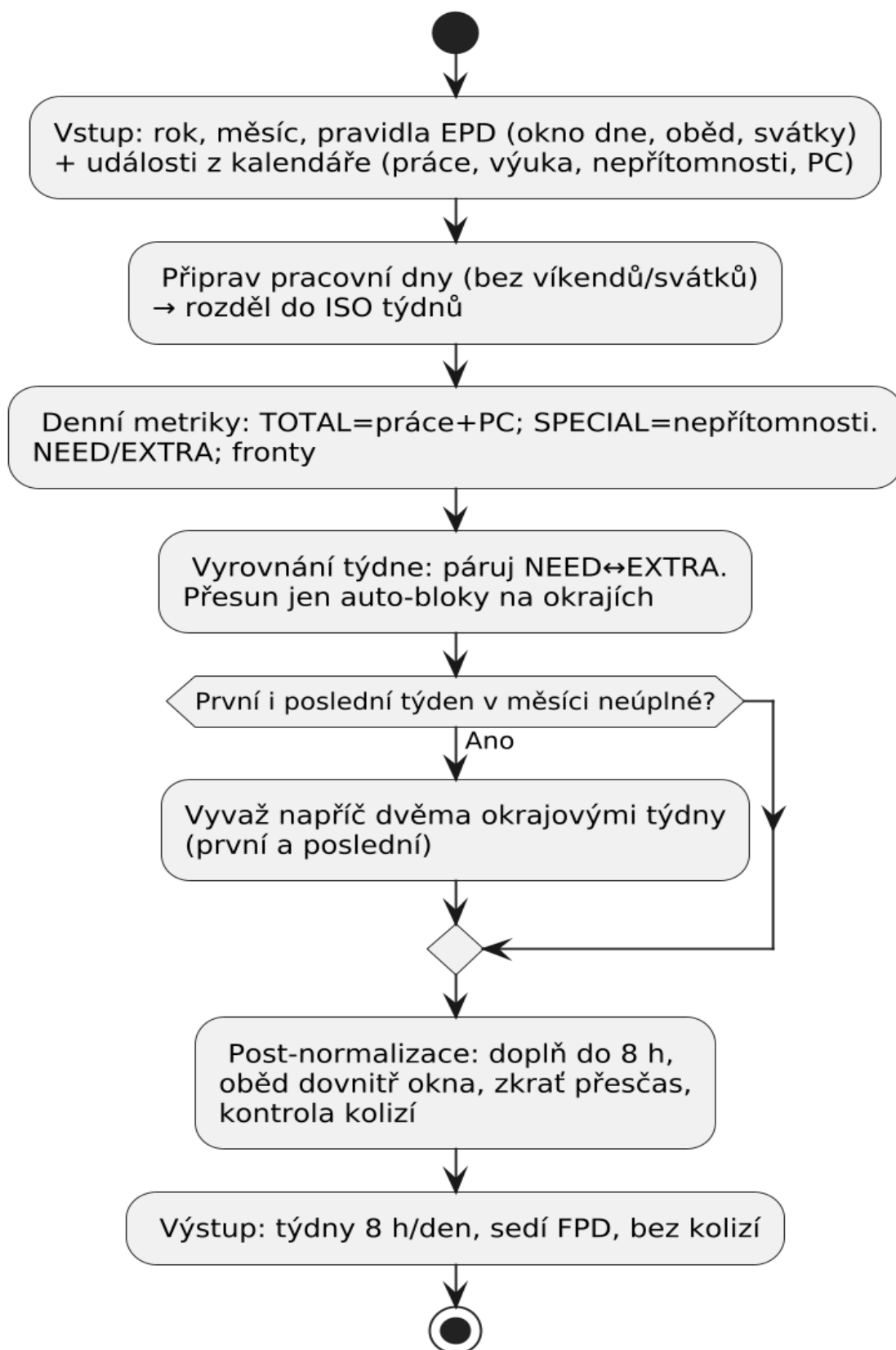
Výpočty pro každý týden

4. Denní metriky (pro každý den):
 1. Práce = běžná práce + služební cesta.
 2. Zvláštní stavy (dovolená, nemoc, OČR, lékař) mají přednost: pokud překrývají výuku/práci, překryv se nepočítá do práce.
 3. Spočti $TOTAL = (\text{práce vč. služební cesty}) + (\text{započitatelné zvláštní stavy})$.
 4. Spočti $NEED = \text{kolik chybí do 8 h}$; $EXTRA = \text{o kolik je přes 8 h}$.
5. Párování deficitů a přebytků:
 1. Vytvoř dvě fronty: dny s $NEED$ a dny s $EXTRA$.
 2. Ber dvojice (dárce s $EXTRA \rightarrow$ příjemce s $NEED$).
 3. Přesun času dělej jen auto-bloky na okrajích dne:
 1. u dárce zkracuj konec, pak začátek (jen auto-bloky),
 2. u příjemce prodlužuj konec, pak začátek (opět auto-bloky).
 4. Po každém zásahu regeneruji auto-události (aby seděly okraje) a srovnej oběd dovnitř pracovního okna.
6. Když dojdou porovnatelné dvojice:
 1. Zkus „flexibilní dny“, kde existuje auto-blok na okraji; ty můžeš lehce prodloužit/zkrátit, pokud je to potřeba k doladění 8 h.
7. Uzavřenost týdnů:
 1. Plné týdny vyrovnávej jen uvnitř stejného týdne.
 2. Výjimka: pokud jsou první i poslední týden v měsíci neúplné, můžeš je vyrovnávat mezi sebou (berou se jako jeden logický celek okrajů měsíce).

Post-normalizace měsíce

8. Dopln chybějící minuty do 8 h (jen pokud to jde přes okrajové auto-bloky).
9. Vynucuj oběd: oběd musí ležet uvnitř pracovního okna (příchod–odchod).

1. Když se oběd nikam nevejde, zkus jej „vyříznout“ z vnitřního auto-bloku (vytvořit uprostřed volno odpovídající délky).
10. Zkrat' denní přesčas (pokud zbyl) opět jen auto-bloky na okrajích.
11. Kontrola: žádné kolize, týdny dávají 8 h/den, sedí měsíční fond.



Obrázek 14 Blokový diagram algoritmu

2.7.2 Algoritmus importu rozvrhu ze STAG

Cíl je vytvořit manuální události práce pro konkrétní dny a časy z CSV souboru, uložit je do DB a doplnit den o automatické bloky/oběd tak, aby se dal hned používat pro EPD.

Vstup a výstup

1. Vstup: cesta k CSV (kódování Windows-1250, oddělovač, hodnoty v uvozovkách)
2. Výstup: seznam vytvořených událostí Event typu Work (ne-auto).

Algoritmus ze souboru čte následující věci:

1. Datum dne v týdnu (základní den pro výuku).
2. Rozsah platnosti: *od data ... do data*.
3. Čas od/čas do (začátek a konec hodiny).
4. Rozsah týdnů (*weekFrom...weekTo*) a parita:
 1. L = liché týdny, S = sudé týdny, J = každý druhý týden se posunem proti *weekFrom* (tj. bere „každý druhý“, ale nezačíná prvním v rozsahu), K = každý týden.
5. Název a popis (předmět, místnost ap.).

Dále následuje popis algoritmu krok za krokem.

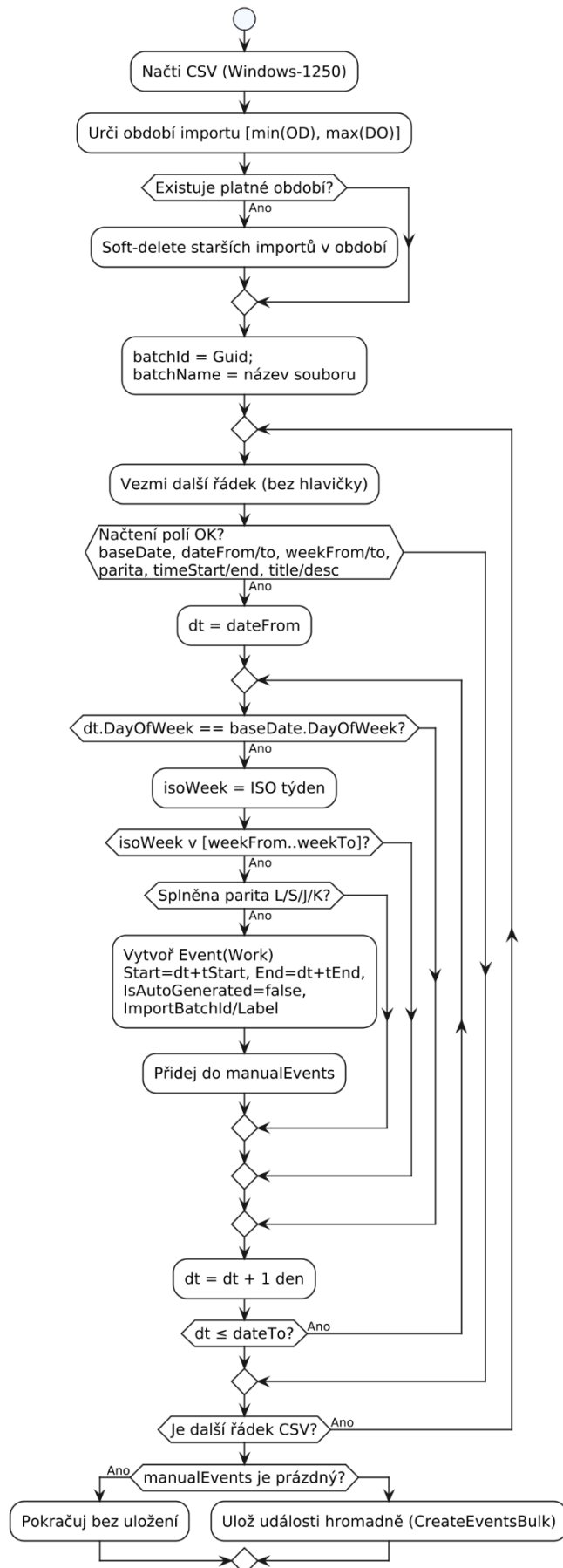
1. Projde všechny řádky, zjistí nejmenší datum „od“ a největší datum „do“. V databázi provede soft-delete všech dříve importovaných událostí, které spadají do tohoto období. (Metoda *BulkSoftDeleteImportedInRangeAsync*.)
2. Prochází všechny řady a vytváří události. Z řádku se načte: *baseDate* (den v týdnu), *dateFrom/dateTo*, *timeStart/timeEnd*, *weekFrom/weekTo*, *parita*, *title/description*. Pak se udělá smyčka přes dny v intervalu *dateFrom...dateTo*:
 1. vezmi jen dny se stejným dnem v týdnu, jako má *baseDate*.
 2. spočti ISO týden a zkontroluj, že je v rozsahu *weekFrom...weekTo*.
 3. použij paritu (L, S, J, K) a případně den přeskoč.
 4. pokud prošel, vznikne manuální událost Event
 1. *StartTime* = den + *timeStart*, *EndTime* = den + *timeEnd*.
 2. *EventType* = Work, *IsAutoGenerated* = false,
 3. nastaví se *ImportBatchId* a *ImportLabel* (název souboru).Vše se sbírá do seznamu *manualEvents*
3. Pokud je seznam neprázdný, uloží se do DB (*CreateEventsBulk*)
4. Pro každý den, kterého se import dotkl:

1. *RegenerateDailyEventsAsync(day)*
→ regeneruje auto-bloky (okraje dne) a doplní/zarovná oběd; řeší případné kolize (přes předaný callback *_askCollision*).
2. *AdjustUserSettingsForDay(day)*
→ najde první start práce a poslední konec práce (počítá i služební cesty), nastaví *Arrival/Departure*; oběd vezme z existujícího „Lunch“ eventu, jinak z globálního výchozího času (pondělní oběd).
3. *TrimOvertimeByAutoBlocksAsync(day)*
→ pokud má den víc než 8 h, zkrátí jen automatické okrajové bloky (manuální výuku nechává).
5. Odešle zprávy *AutoEventsGeneratedMessage* a *EpdGeneratedMessage*, aby se obrazovky obnovily.
6. Metoda vrátí seznam vytvořených manuálních událostí (to, co přišlo z rozvrhu).

Některé poznámky a omezení algoritmu:

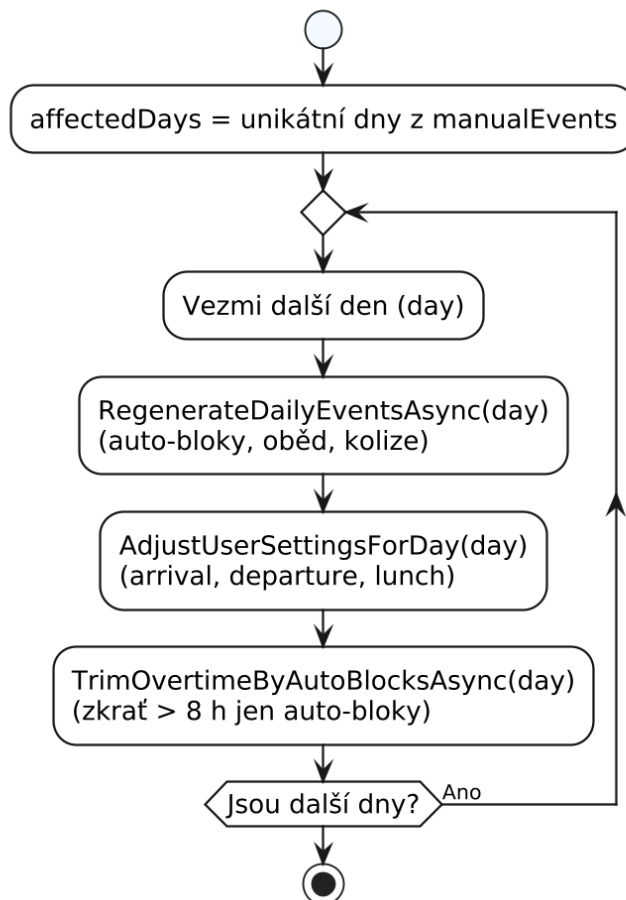
- Před importem se staré importy v daném období jen označí jako smazané (soft-delete), aby šel import bezpečně zopakovat.
- Import vytváří jen manuální „Work“ události; vše kolem oběda a drobných dorovnání řeší automatický generátor po uložení.
- Víkendy a svátky filtruje generátor/ostatní služby podle kalendáře; do importu se běžně nedostanou.
- Helper *LoadTeacherSchedule(...)* je alternativní čtečka CSV (robustnější mapování), ale v hlavním importu se nepoužívá.
- Nastavení oběda v *AdjustUserSettingsForDay* padá na globální pondělní časy, pokud v dni žádný „Lunch“ není — to je záměrné zjednodušení a dá se později zpřesnit (např. podle dne v týdnu).

Import rozvrhu ze CSV - část 1/3: Načtení a filtrování



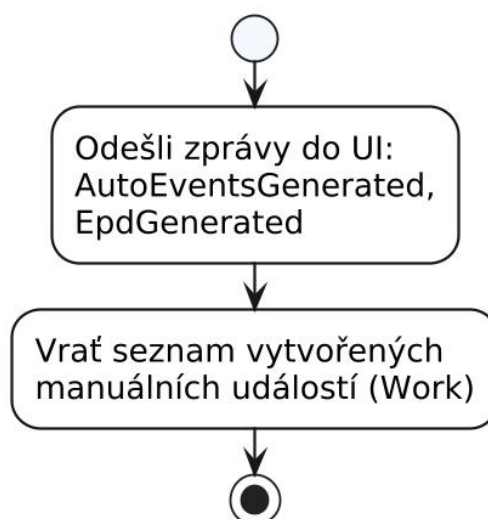
Obrázek 15 Blokový diagram algoritmu Importu 1/3

Import rozvrhu ze CSV - část 2/3: Zpracování dotčených dnů



Obrázek 16 Blokový diagram algoritmu Importu 2/3

Import rozvrhu ze CSV - část 3/3: Notifikace a návrat



Obrázek 17 Blokový diagram algoritmu Importu 3/3

3 Testování aplikace

3.1 Cíl a kritéria přijetí

Cíl: ověřit, že aplikace správně:

- načítá rozvrh z CSV (STAG),
- ukládá a přepočítává události podle pravidel EPD,
- vyrovnává hodiny v týdnech i mezi okrajovými týdny měsíce,
- vytváří PDF s tabulkou EPD,
- funguje na Windows i Linuxu.

Kritéria přijetí:

- Funkčnost: všechny níže uvedené scénáře proběhnou bez chyby; týdenní i měsíční součty sedí.
- Křížová platforma: stejné funkce jsou dostupné na Windows i Linuxu; obsah PDF je totožný (texty, součty, počet řádků).
- Stabilita: chybné vstupy nevedou k pádu; uživatel dostane jasnou zprávu.
- Odezva: UI zůstává plynulé během importu i exportu

3.2 Testovací prostředí

Aplikace byla testována na následujících operačních systémech a sadách nástrojů

- Windows: Windows 11 (23H2), .NET 8, Avalonia UI, SQLite (soubor)
- Linux: Ubuntu 22.04 LTS (GNOME, Wayland/X11), .NET 8, Avalonia UI, systémové fonty (DejaVu/Liberation),

3.3 Metodika

- Integrační testy: řetězec Import → auto-události → vyrovnání → PDF nad skutečnou SQLite.
- Systémové/ruční testy: scénáře v GUI, klikací průchod obrazovkami.
- Kontrola PDF: počet stran, hlavičky, zvýraznění svátků, součty týdnů i měsíce. Binární shoda souboru se nevyžaduje.

3.4 Očekávané rozdíly Windows × Linux

- Fonty a metriky: Linux používá jiné fonty. Může se lišit šířka sloupců a zalomení textu.
- Vzhled prvků: tloušťka rámečků a scrollbarů se liší.
- Požadavek: nic se nesmí překrývat, text musí být čitelný, ovládací prvky musejí být dostupné. Obsah PDF (texty a součty) musí být stejný.

3.5 Scénáře

1. Import správného CSV.
2. Opakovaný import (přepis dřívější šarže přes soft-delete).
3. Špatné kódování vstupu.
4. Chybějící povinný sloupec.
5. Překrývající se výuka v jeden den.
6. Neúplné týdny (svátky, začátek/konec měsíce).
7. Vyrovnání mezi prvním a posledním neúplným týdnem měsíce.
8. Služební cesta + výuka v jeden den.
9. Krátká nepřítomnost překryje výuku.
10. Ruční úprava události a regenerování auto-bloků.
11. Export PDF.
12. Rychlost importu.
13. Rychlost PDF.
14. Škálování UI (125–150 %).
15. Není právo zápisu (DB/PDF do chráněné složky).
16. Záznamy mimo sledovaný měsíc.
17. Zrušení dlouhé operace v UI.
18. Stabilita: opakovaný import stejného souboru.

3.6 Souhrnná tabulka testů

ID	Platforma	Scénář	Akce systému (zkráceně)	Výsledek testu
1	Win/Linux	Import správného CSV	Načte CSV → určí období → soft-delete staré šarže → vytvoří Work události → pro dotčené dny vygeneruje auto-bloky a oběd, zkrátí >8 h jen auto-bloky	PASS – dny vyplněny, součty sedí, bez chyb
2	Win/Linux	Opakovaný import	Najde stejné období a označí staré záznamy jako smazané → uloží novou šarži	PASS – bez duplicit; v DB je jen aktuální šarže
3	Win/Linux	Špatné kódování	Nepodaří se číst datum/čas → vrátí řízenou chybu a zobrazí zprávu	PASS – aplikace nespadne; uživatel dostane vysvětlení
4	Win/Linux	Chybí sloupec	Kontrola délky a povinných indexů → řádky přeskočí, zaloguje varování	PASS – import doběhne; problémové řádky uvedeny
5	Win/Linux	Překryvy výuky	Při výpočtu se překryvy nepočítají dvakrát; kolize jsou označeny	PASS – za den nejvýše 8 h práce; kolize viditelné
6	Win/Linux	Neúplné týdny	Požadavek týdne = 8 h × počet pracovních dnů (svátky/volno odečteny)	PASS – týden se svátkem = 32 h; zač./konec měsíce v pořádku
7	Win/Linux	Vyrovnaní okrajů měsíce	Spustí párové vyrovnaní mezi prvním a posledním neúplným týdnem	PASS – přenos hodin proběhl; plné týdny beze změny
8	Win/Linux	Služební cesta + výuka	PC se započítá jako práce; překryv s výukou se nezdvouje	PASS – den max. 8 h; priority dodrženy
9	Win/Linux	Nepřítomnost překryje výuku	Do výkazu se zapíše nepřítomnost; práce v překryvu se nepočítá	PASS – „Neodpracováno“ s důvodem; součty sedí
10	Win/Linux	Ruční úprava	Po editaci se znovu vygenerují auto-bloky a oběd; uloží se UserSettings	PASS – okraje dne a oběd validní; bez kolizí
11	Win/Linux	Export PDF	Vytvoří tabulku za měsíc se součty a svátky	PASS – obsah PDF sedí na obou OS
12	Win/Linux	Rychlost importu	Import měsíce ze souboru ok.csv	PASS – UI neblokuje; čas <5 s
13	Win/Linux	Rychlost PDF	Export měsíce do PDF	PASS – generace <3 s; soubor lze otevřít

14	Win/Linux	Škálování UI	Zvětšení 125–150 %	PASS – nic se nepřekrývá; text čitelný
15	Win/Linux	Bez práv zápisu	Pokus uložit DB/PDF do chráněné složky	PASS – zobrazeno srozumitelné hlášení; nabídnuta změna cesty
16	Win/Linux	Data mimo měsíc	Při tisku se ignorují dny mimo zvolený měsíc	PASS – v PDF nejsou navíc dny; součty správně
17	Win/Linux	Zrušení operace	Přerušování importu v UI	PASS – bezpečné ukončení; DB v pořádku
18	Win/Linux	Stabilita/determinismus	Opakovaný import stejného CSV	PASS – výsledek stejný; bez rozdílů

3.7 Závěr a známá omezení

- Import, vyrovnání i export PDF se chovají shodně na Windows i Linuxu.
- Vizuelní rozdíly na Linuxu jsou malé a nemají vliv na čitelnost ani na data.
- PDF soubory se mohou lišit v metadatech (čas vytvoření). Proto porovnáváme obsah, nikoli binární soubor.

ZÁVĚR

Cílem této práce bylo navrhnout, vytvořit a ověřit multiplatformní desktopovou aplikaci, která usnadní evidenci pracovní doby vyučujících. Aplikace umí načíst rozvrh ze systému STAG, zpracovat jej podle pravidel EPD a vygenerovat přehledný PDF dokument připravený k odevzdání.

V teoretické části jsem stručně vysvětlil použité technologie: jazyk C# a platformu .NET, databázi SQLite a deklarativní popis rozhraní v AXAML/Avalonia UI. Dále jsem popsal vývojové prostředí (Microsoft Visual Studio a SQLiteStudio), architekturu na vzoru MVVM a hlavní knihovny z NuGetu (Avalonia, ReactiveUI, Entity Framework Core, QuestPDF aj.). Tyto volby zajišťují přenositelnost, čisté oddělení logiky a vzhledu a dobrou testovatelnost.

V praktické části jsem ukázal strukturu databáze (Events, GlobalSettings, UserSettings), rozdělení projektových částí a implementaci GUI v Avalonia UI. Klíčovou roli hraje algoritmus vyrovnávání hodin, který při generování náhledu a exportu upraví pracovní úseky v povolených mezích tak, aby seděly týdenní i měsíční součty a aby rozvrh zůstal srozumitelný.

Hotová aplikace nabízí jednoduché ovládání, zobrazení dne/týdne/měsíce a okamžitý náhled PDF. Díky Avalonii běží na Windows i Linuxu. SQLite s EF Core poskytují spolehlivé lokální ukládání dat.

Řešení zkracuje čas potřebný pro měsíční a týdenní přehledy a snižuje riziko chyb. Tím zvyšuje efektivitu a transparentnost správy pracovního fondu na katedrách a fakultách.

Do budoucna lze přidat jednotné přihlášení (SSO), auditní logování a další exportní formáty. Tyto kroky dále posílí použitelnost i škálovatelnost aplikace.

POUŽITÁ LITERATURA

1. **MICROSOFT**, 2025. *A tour of the C# language*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/csharp/tour-of-csharp/overview>
2. **MICROSOFT**, 2025. *.NET – Build modern apps and services* (úvod, multiplatformnost). In: **dotnet.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://dotnet.microsoft.com/en-us/>
3. **AVALONIA UI**, 2025. *The MVVM Pattern*. In: **docs.avaloniaui.net** [online]. Avalonia, 2025 [cit. 2025-08-24]. Dostupné z: <https://docs.avaloniaui.net/docs/concepts/the-mvvm-pattern/>
4. **REACTIVEUI COMMUNITY**, 2025. *Getting Started*. In: **reactiveui.net** [online]. ReactiveUI, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.reactiveui.net/docs/getting-started/>
5. **SQLITE**, 2024. *SQL As Understood By SQLite*. In: **sqlite.org** [online]. SQLite, 2024 [cit. 2025-08-24]. Dostupné z: <https://www.sqlite.org/lang.html>
6. **QUESTPDF**, 2025. *QuestPDF – C# PDF Generation Library*. In: **questpdf.com** [online]. QuestPDF, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.questpdf.com/>
7. **ELECTRON**, 2025. *What is Electron?* In: **electronjs.org/docs/latest** [online]. Electron, 2025 [cit. 2025-08-24]. Dostupné z: <https://electronjs.org/docs/latest/>
8. **PYTHON SOFTWARE FOUNDATION**, 2025. *Python 3.13 documentation*. In: **docs.python.org** [online]. PSF, 2025 [cit. 2025-08-24]. Dostupné z: <https://docs.python.org/>
9. **THE QT COMPANY**, 2025. *Qt for Python (PySide6) – Official documentation*. In: **doc.qt.io** [online]. Qt, 2025 [cit. 2025-08-24]. Dostupné z: <https://doc.qt.io/qtforpython-6/>
10. **JETBRAINS**, 2025. *Compose Multiplatform – Get Started*. In: **jetbrains.com** [online]. JetBrains, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.jetbrains.com/help/kotlin-multiplatform-dev/compose-multiplatform.html>
11. **MICROSOFT**, 2023. *ASP.NET MVC Overview – Model–View–Controller*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/overview/asp-net-mvc-overview>

12. **REACTIVEUI COMMUNITY**, 2025. *The Zen of the ViewModel*. In: **reactiveui.net** [online]. ReactiveUI, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.reactiveui.net/docs/handbook/view-models/>
13. **CMU/SEI**, 2010. *What Is Your Definition of Software Architecture?* In: **sei.cmu.edu** [online]. Carnegie Mellon University, 2010 [cit. 2025-08-24]. Dostupné z: <https://www.sei.cmu.edu/library/what-is-your-definition-of-software-architecture/>
14. **MICROSOFT**, 2023. *Data binding a chování v Avalonia a WPF (Behaviors)*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/desktop/wpf/advanced/behaviors-overview>
15. **AVALONIA XAML BEHAVIORS**, 2024. *GitHub – Avalonia.Xaml.Behaviors*. In: **github.com/AvaloniaXAMLBehaviors** [online]. Avalonia Team, 2024 [cit. 2025-08-24]. Dostupné z: <https://github.com/AvaloniaXAMLBehaviors>
16. **AVALONIA COMMUNITY**, 2024. *Behaviors v Avalonia UI*. In: **docs.avaloniaui.net** [online]. Avalonia Community, 2024 [cit. 2025-08-24]. Dostupné z: <https://docs.avaloniaui.net/docs/controls/behaviors>
17. **MICROSOFT**, 2023. *Common web application architectures – monolitická aplikace*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>
18. **MICROSOFT**, 2025. *Microservices architecture style – přehled a trade-offy*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>
19. **FOWLER, Martin**, 2015. *Monolith First*. In: **martinfowler.com** [online]. 2015 [cit. 2025-08-24]. Dostupné z: <https://martinfowler.com/bliki/MonolithFirst.html>
20. **AZURE ARCHITECTURE CENTER**, 2025. *Architecture styles – přehled*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/azure/architecture/guide/architecture-styles/>
21. **MICROSOFT**, 2024. *Entity Framework Core – SQLite provider*. In: **learn.microsoft.com** [online]. Microsoft, 2024 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/ef/core/providers/sqlite/>
22. **ARTIFEX SOFTWARE**, 2025. *Ghostscript – official documentation*. In: **ghostscript.com** [online]. Artifex Software, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.ghostscript.com/>

23. **HABJAN, John** a kol., 2025. *Ghostscript.NET – .NET wrapper pro Ghostscript*. In: **github.com/jhabjan/Ghostscript.NET** [online]. 2025 [cit. 2025-08-24]. Dostupné z: <https://github.com/jhabjan/Ghostscript.NET>
24. **MICROSOFT**, 2025. *Visual Studio – Product documentation (Overview, IDE features)*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/visualstudio/ide/?view=vs-2022>
25. **SQLTESTUDIO**, 2025. *SQLiteStudio – Official website (Features, Downloads, Docs)*. In: **sqlitestudio.pl** [online]. 2025 [cit. 2025-08-24]. Dostupné z: <https://sqlitestudio.pl/>

SEZNAM PŘÍLOH

Příloha A: Zdrojový kód aplikace TeacherScheduleApp

PŘÍLOHA A: Zdrojový kód aplikace TeacherScheduleApp

Tato příloha obsahuje zip archiv s projektem z Visual Studio 2022 jménem TeacherScheduleApp